



Reading
App Builder

Installing and Building Apps on a Mac



Reading App Builder: Installing and Building Apps on a Mac

© 2021, SIL International

Last updated: 25 March 2025

You are free to print this manual for personal use and for training workshops.

The latest version is available at
<http://software.sil.org/readingappbuilder/resources/>

and on the Help menu of Reading App Builder.

Contents

1. Introduction.....	5
2. Installing Reading App Builder.....	5
3. Installing Prerequisites for Android.....	5
3.1. Java Development Kit (JDK).....	6
3.2. Installing Android Software Development Kit (SDK).....	7
3.2.1. Downloading the Android SDK packages from the internet.....	7
3.2.2. Copying the Android SDK files from someone else.....	10
4. Installing Prerequisites for iOS.....	10
4.1. Install Xcode.....	11
4.2. Verify Xcode Installation.....	11
4.3. Install Transporter.....	11
5. Installing Aeneas.....	12
6. Testing App in iOS Simulator.....	14
6.1. Run the iOS Simulator.....	14
6.2. Installing Additional Simulators.....	15
6.3. Manually Installing Apps into the Simulator.....	15
7. Creating iOS Certificates and Provisioning Profiles.....	15
7.1. Enroll in the Apple Developer Program.....	15
7.2. Create Signing Certificate.....	16
7.3. Create Provisioning Profile.....	17
8. Building an iOS App.....	18
8.1. Application builds available for iOS.....	18
8.1.1. Dedicated App.....	19
8.1.2. Container App.....	19
8.1.3. Asset Package.....	20
8.1.4. Container App Website.....	20

8.2.	App Type (iOS) Tab.....	22
8.3.	IPA Tab.....	23
8.4.	Signing (iOS) Tab.....	24
9.	Testing an iOS App.....	27
10.	Using Xcode to Test an iOS App.....	27
11.	Using DeployGate to Test an iOS App.....	28
11.1.	Creating a DeployGate Account.....	28
11.2.	Uploading your first app.....	28
11.3.	Registering a Device.....	30
12.	Uploading an iOS App to Apple App Store.....	36
13.	Using Test Flight to Test an iOS App.....	37
14.	Apple Privacy Policy.....	38
14.1.	Data Types.....	38
14.2.	Product Interaction.....	38
15.	Building from Terminal.....	38
15.1.	Installing Command Line Interface.....	39
15.2.	Command Line Interface Options.....	39
16.	Using Firebase in an iOS App.....	41
16.1.	Adding an iOS App.....	41
16.2.	iOS Configuration for Firebase in SAB.....	45
16.3.	Security Feature Support in iOS App.....	46
16.4.	Firebase Messaging.....	46
16.5.	Firebase Crashlytics for iOS.....	48

1. Introduction

This document provides information on how to install Reading App Builder and build apps on an Apple macOS system. It is possible to build an Android app using RAB on Windows, Linux or Mac, but if you want to build an iOS app for the iPhone or iPad, you will need to build it using a Mac computer.

App Builder Platform	Build Android Apps	Build iOS Apps
Windows	✓	
Linux	✓	
macOS	✓	✓

Creating an Android app on a Mac is essentially the same process as it is for Windows or Linux. To create a corresponding iOS app, you will need to enter a few more configuration items.

The apps generated by RAB for iOS will run on iPhones and iPads with iOS 12.2 or higher.

2. Installing Reading App Builder

To install the Reading App Builder program files:

1. Download the current Mac installer file (dmg) from the RAB website:
<http://software.sil.org/readingappbuilder/download/>
2. Double click on the file within **Finder** to open the disk image that contains the Reading App Builder application.
3. Copy the **Reading App Builder** application to your **Application** folder. This can be done by dragging the Reading App Builder icon from the disk image window to the shortcut of the Applications folder in the same window.

3. Installing Prerequisites for Android

If you want to build Android apps, you need to install the following components on your computer:

1. Java Development Kit (JDK)
2. Android Software Development Kit (SDK)

3.1. Java Development Kit (JDK)

You will need version 17 of the Java Development Kit (JDK) to build apps. We recommend you use Zulu, which is a free distribution of OpenJDK from Azul.

1. Go to the **Download Zulu Builds of OpenJDK** website:

<https://www.azul.com/downloads/?version=java-17-lts&os=macos&architecture=arm-64-bit&package=jdk-fx#zulu>

There are many downloads on this page, but the above link will filter the ones you see (Java Version: Java 17 LTS; Operating System: MacOS; Architecture: ARM 64-bit; Java Package: Java FX).

2. Scroll down the page until you see the downloads under the heading **Download Azul Zulu Builds of OpenJDK**:

The screenshot shows the Azul Zulu Builds of OpenJDK website. At the top, there are two main cards: 'Azul Zulu Builds of OpenJDK' and 'Azul Prime'. The 'Azul Zulu Builds of OpenJDK' card has a 'Download Free (.dmg, 227MB)' button and mentions '25.0.2+10 (latest LTS) macOS M1/M2 64-bit, JDK'. The 'Azul Prime' card has a 'Request a Trial' button and mentions 'Fastest JVM in the world'. Below these cards, there is a navigation bar with 'Azul Zulu', 'Package Managers / API', and 'Platform Components'. Under 'Azul Zulu', there are filters for 'Java Version' (Java 17 (LTS)), 'Operating System' (macOS), 'Architecture' (ARM 64-bit), and 'Java Package' (JDK FX). There is also a 'Reset Filters' button and a checkbox for 'Include older versions'. Below the filters, there is a table of download links for 'Java 17 (LTS)'. The table has columns for 'Java Version', 'Operating System', 'Architecture', 'Java Package', and 'Download'. The first row shows '17.0.18+8' for 'macOS', 'ARM 64-bit v8', 'JDK FX', and a 'Download' button.

Java Version	Operating System	Architecture	Java Package	Download
17.0.18+8 Azul Zulu: 17.64.17	macOS	ARM 64-bit v8	JDK FX	Download

1. You have a choice between two different architectures: **x86 64-bit** and **ARM 64-bit**. You can find the processor type of your machine by clicking on the Apple symbol at the top left of your screen and selecting **About This Mac**. If you have an older Mac

running with an Intel x86 chip, select x86 (64 bit) otherwise you will need the ARM 64-bit.

Once you have identified your device architecture, you have a choice between a **dmg** file, a **tar.gz** file and a **zip** file. **Download the .dmg file** since it comes with its own installer program.

The file you download will have a filename something like this:

zulu17.64.17-ca-fx-jdk17.0.18-macosx_x64.dmg

2. **Double-click the file in Finder** and follow the instructions in the installation wizard to install it. By default, the installer will install the JDK to the following folder:

/Library/Java/JavaVirtualMachines/zulu-17.jdk/Contents/Home

Important: If you change the JDK install folder to something other than the default folder, you will need to remember the location of the folder so you can tell Reading App Builder where to find the JDK.

3.2. Installing Android Software Development Kit (SDK)

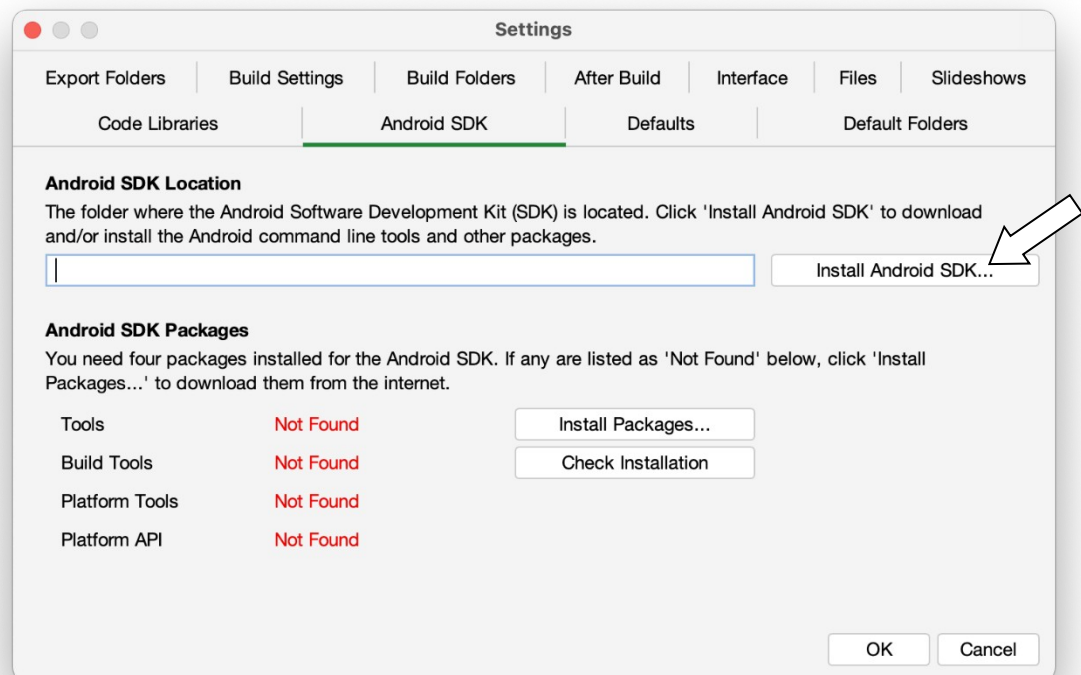
The Android Software Development Kit (SDK) is needed for building Android apps. There are two ways of installing the Android SDK:

1. **Online: Download the Android SDK packages from the internet:**
Use the Android SDK Installation wizard to download and install the command line tools and three additional packages. This method will require an internet connection.
See 3.2.1 for more details.
2. **Offline: Copy the Android SDK files from someone else:**
If you know someone who has already downloaded and installed the Android SDK, you can copy all the files from them.
This method is especially useful in a training workshop where several people need to install the SDK but have limited internet bandwidth.
See 3.2.2 for more details.

3.2.1. Downloading the Android SDK packages from the internet

To install the Android SDK from the internet:

1. Launch **Reading App Builder**.
2. Select **Reading App Builder Ø Preferences** from the main menu.
3. Go to the **Android SDK** tab, which is the second tab.
4. Click the **Install Android SDK** button.



5. Follow the instructions on each page of the **Install Android SDK** wizard to download each of the Android SDK packages and install them.

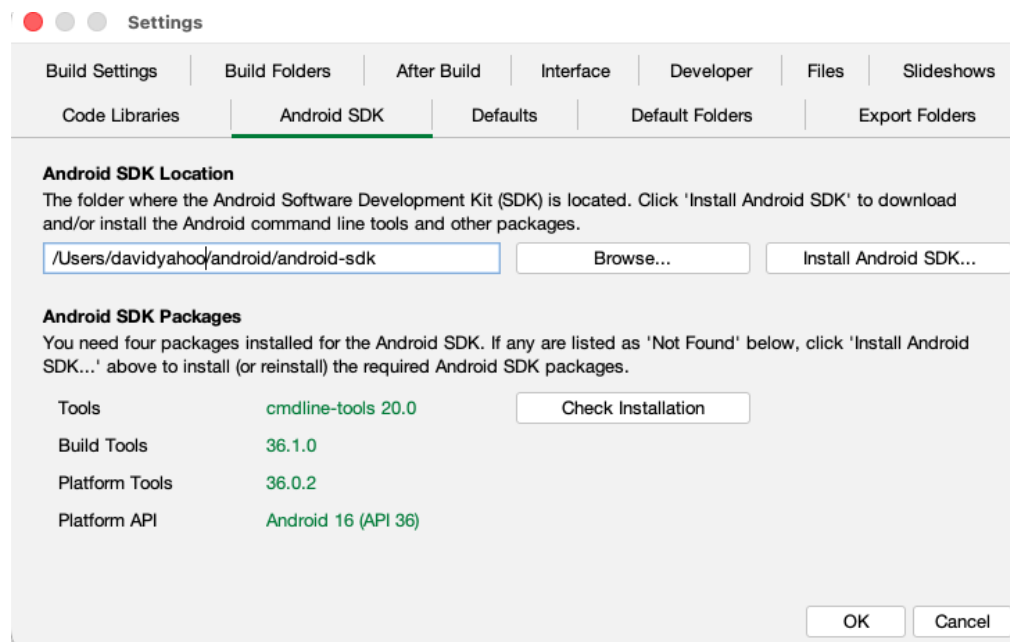
When you are asked to specify a target folder, a good place is:

/Users/your-name/Android-SDK.

Four packages will be downloaded and installed:

- Command line tools,
- Build Tools,
- Platform Tools, and
- Platform API.

If the installation was successful, you will see the version numbers displayed in green.



If any of the Build Tools, Platform Tools or Platform API is listed as “Not Found” (displayed in red), click the **Install Packages** button to install them.

Click the **Check Installation** button to confirm that all the packages have been installed correctly.

You can skip section 3.2.2 and go straight to section 4.

3.2.2. Copying the Android SDK files from someone else

If you know someone who has already downloaded and installed the Android SDK and is successfully building apps with it, you can copy all of their Android SDK files to a folder on your computer.

You need to look for the top-level Android SDK folder, such as **/Users/user-name/Android-SDK**, and copy the whole folder and its contents to your computer. A location such as **/Users/your-name/Android-SDK** is good. If it makes it easier, you can zip the folders and then unzip them onto your computer.

Note that there is no setup program to run. Copying the files from one computer to another is sufficient.

Tip: A typical Android SDK folder can be quite large (over 1 GB, depending on which additional packages have been installed). To build an app, you do not actually need all of the Android SDK files. If you want to cut down the number of files, here is a list of the essential and optional folders:

Android SDK Folder	Required for building apps?
cmdline-tools (or tools)	Yes
build-tools	Yes (you only need the sub-folder for the latest version)
platforms	Yes (you only need android-30 for now)
platform-tools	Yes
add-ons	No
docs	No
emulator	No, unless you want to use an emulator
extras	No
licenses	Yes
sources	No
system-images	No, unless you want to use an emulator
temp	No

4. Installing Prerequisites for iOS

If you want to build iOS apps and upload them to the Apple App Store, you need to install the following components:

1. Xcode
2. Transporter

4.1. Install Xcode

Xcode is required to build iOS Apps. To install Xcode, simply search for Xcode in the Mac App Store and install it. Open Xcode at least once to agree to the licensing restrictions and install components.



4.2. Verify Xcode Installation

To verify that you have successfully installed Xcode and that it is will be correctly used by Reading App Builder, open Terminal and run the following command to print the path to the active developer directory:

```
xcode-select -p
```

```
$ xcode-select -p
/Applications/Xcode.app/Contents/Developer
```

If you have had Xcode Command-line Tools installed previously, it might still be pointing to that installation directory and Reading App Builder will not work correctly.

```
$ xcode-select -p
/Library/Developer/CommandLineTools
```

To correct this situation, run the following command to set the active developer directory.

```
sudo xcode-select -s /Applications/Xcode.app/Contents/Developer
```

4.3. Install Transporter

Transporter is used to upload iOS apps to App Store Connect. To install Transporter, simply search for Transporter in the Mac App Store and install it.

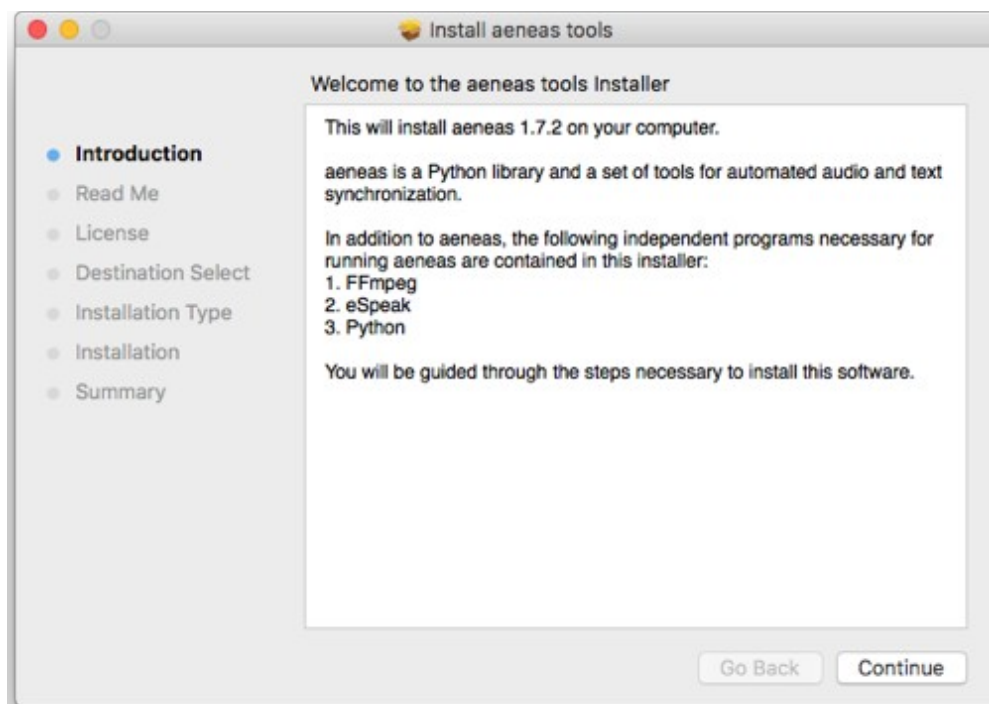


5. Installing Aeneas

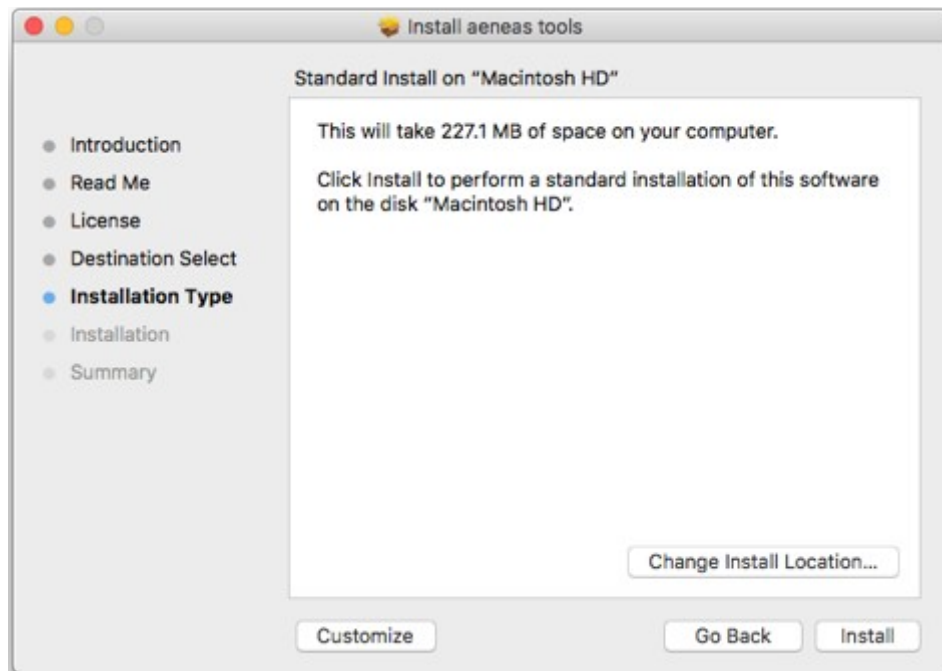
Aeneas is the audio-text synchronization tool that may be run from within Reading App Builder to create timing files for phrase-by-phrase highlighting. If the apps you are building do not include audio or if the timing files are already available, then there is no need to install it.

To install **aeneas**:

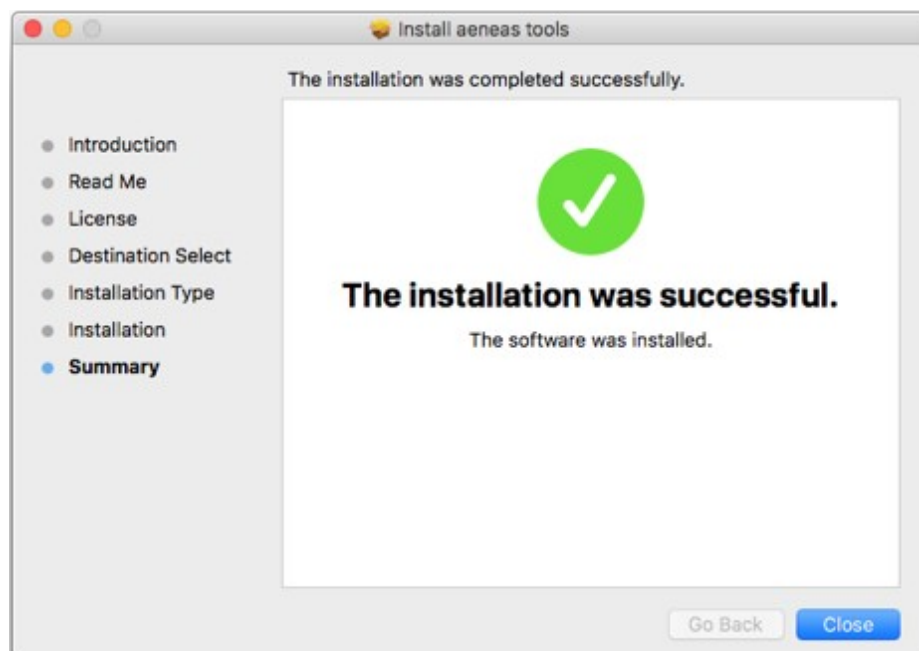
1. Download the **aeneas tools** for Mac file (dmg) from the RAB website from the section labeled **Audio Synchronization Tools**:
<http://software.sil.org/readingappbuilder/download/>
2. Double click on the file within Finder to open the disk image that contains the aeneas-mac-setup install package. Control-click on the install package and select **Open**. You will get a warning that this package is from an unidentified developer. Press **Open**.
3. The introduction screen will display. Press **Continue**.



4. A “Read Me” screen will display next followed by a “License” screen. Press **Continue** on both screens.
5. Pressing the **Continue** button on the License screen will bring up a screen asking if you agree to the terms of the license. Press **Agree**.
6. The “Destination Select” screen selects the default drive. Press **Continue**.
7. The “Installation Type” screen displays next for a standard install. Press **Install**.



8. The installer will prompt for credentials to install the software. Enter a username and password with permissions and press **Install Software**.
9. At this point the installation will start and will show progress screens until it completes. A terminal window will popup briefly to test the installation. When the application completes successfully, the original screen will show:



10. Press **Close**.

Aeneas is installed in `/usr/local/lib/python2.7/site-packages`.

6. Testing App in iOS Simulator

When you want to test your app, you can either use a device or a simulator. To test with a device, you will need a signing certificate and provisioning profile (see next section). To test with a simulator, you will need to download Xcode, the integrated development environment used to build and test iOS and Mac apps. Xcode is available for free in the Mac App Store and is quite large (5.46 GB). RAB requires Xcode 26 or greater (which requires macOS Sequoia).

To install:

1. Go to <https://itunes.apple.com/us/app/xcode/id497799835?mt=12> or search for “Xcode” in the App Store app on macOS.
2. Install from the App Store
3. Start Xcode at least once to complete the installation

6.1. Run the iOS Simulator

Once you have a project configured and ready to test, click on the **Run iOS App in Simulator** on the toolbar.



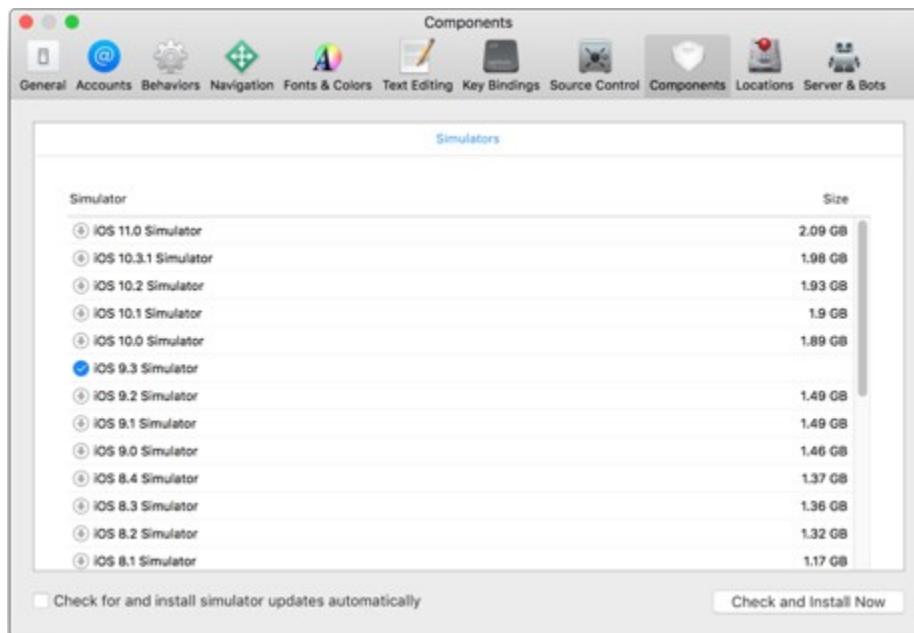
Select the simulator you would like to run on and click **Start**. It takes a little bit of time for the simulator to start. If you want to switch simulators, select a different Simulator from the **Simulator Type** combo box and click **Start** again. It will close the previous Simulator and start the new one.

Select the project you want to test (it defaults to the selected project in the Apps list) and click on **Build**. This will build the app for the Simulator in a separate terminal window. When the build is complete, you can click on **Launch** to run the app in the Simulator.

You may close the dialog and make changes to your project. When you restart the Run iOS Simulator dialog again, you will need to Build and Launch again for the changes to be included.

6.2. Installing Additional Simulators

You may install simulators for previous versions of iOS by launching Xcode and viewing the preferences dialog and selecting the Components tab.



6.3. Manually Installing Apps into the Simulator

If there is some problem with launching the simulator from the **Run iOS Simulator** dialog, you can manually install the app by dragging the built app from the Simulator output folder (found in ~/App Builder/Reading Apps/Sim Output) onto the Simulator. To start the Simulator, launch **Xcode** and from the **Xcode** menu select **Open Developer Tool > Simulator**. You will still need to rebuild the app from the **Run iOS Simulator** dialog.

7. Creating iOS Certificates and Provisioning Profiles

7.1. Enroll in the Apple Developer Program

To build iOS apps and distribute them through the Apple App Store, you will need to be enrolled in the Apple Developer Program. You can do this as either (i) an individual, or (ii) an organisation. The cost is USD \$99 per year.

To enroll:

1. Go to <https://developer.apple.com/programs/enroll/>
2. Press the **Start Your Enrollment** button to start.

7.2. Create Signing Certificate

When you create an iOS app, it needs to be signed with a certificate.

To create a certificate:

1. Go to the [Apple Developer](https://developer.apple.com) website and log in to your account if you are not already.
2. Select **Certificates, Identifiers & Profiles**.
3. Click the  button to the right of the **Certificates** header.
4. On the **Create a New Certificate** page, select **Apple Distribution**. Then press the **Continue** button.
5. Upload a Certificate Signing Request. Press the **Learn more** link for instructions on how to use Keychain Access on your Mac computer to complete this task. Then press the **Continue** button.
6. On the **Download Your Certificate** page, press the **Download** button to download the certificate (distribution.cer) to your Mac.
7. Open the Keychain Access application. Choose the **login** item from the Keychains section on the left. Choose the **File > Import Items...** menu item and browse to the Downloads folder and select the certificate (distribution.cer).

Now that the certificate is installed in the Keychain, you will be able to access it from within Reading App Builder.

Note: To work with certificates, you will need the Apple Worldwide Developer Relations (WWDR) Certification Authority. It should have been installed with Xcode. When viewing your certificate in the Keychain Access application, if there is an error stating the certificate is not trusted, then install the certification authority.

To get the certification authority:

1. Download the Apple WWDR Certification Authority from:
<https://developer.apple.com/certificationauthority/AppleWWDRCA.cer>
For signing certificates created after January 28, 2021:
<https://www.apple.com/certificateauthority/AppleWWDRCA3.cer>
2. Open the Keychain Access application. Choose the **System** item from the Keychains section on the left. Choose the **File > Import Items...** menu item and browse to the Downloads folder and select the certification authority (AppleWWDRCA.cer). You will be prompted for a username and password that has admin privileges in order to modify the **System** Keychain.
3. Choose the **File > Lock Keychain "System"** menu item.

7.3. Create Provisioning Profile

Distribution provisioning profiles are used to for two primary purposes:

- AdHoc - to install your app on a limited number of registered devices for testing.
- App Store - to submit your app to the App Store.

Creating a provisioning profile

To create a new AdHoc provisioning profile, you will need an App ID and at least one registered device. To create a new App Store provisioning profile, you will need just the App ID.

To create an App ID:

1. Go to the Apple Developer website and log in to your account if you are not already.
2. Select **Certificates, Identifiers & Profiles**.
3. Select **Identifiers** on the left of the page.
4. Click the  button to the right of the **Identifiers** header.
5. Select **App IDs** and click the **Continue** button.
6. Select **App** and click the **Continue** button
7. Enter a **Description** of your choice.
It can be the **App Name** from the **App > Package** page of Reading App Builder.
8. For Bundle ID, choose **Explicit** and enter your app package name from the **App > Package** page of Reading App Builder.
9. Of the App Capabilities, there are only two that may need to be checked. Check the Push Notifications capability if the app uses Firebase Cloud Messaging or Daily Reminders. Check the Associated Domain capability if using Deep Linking.
10. Click the **Continue** button.

To register a device (i.e. a specific iPhone or iPad for testing):

1. Select **Devices** on the left of the page.
2. Click the  button to the right of the **Devices** header.
3. In the section **Register a Device**, enter a name of the device (such as “John Smith’s iPhone”) and its UDID (unique device identifier, a sequence of 40 letters and numbers). Press **Continue**.
4. On the next page, check that the device information is displayed correctly and click **Register** to confirm.

Note that you can register up to 100 devices of each type (e.g. up to 100 iPhones, 100 iPads) per year of your Apple Developer Program membership. You can remove devices that you no longer need at the beginning of the next membership year.

To create a provisioning profile:

1. Select **Profiles** on the left of the page.
2. Click the  button to the right of the **Profiles** header.
3. On the **Register a New Provisioning Profile** page, under **Distribution**, select **Ad Hoc** (for testing) or **App Store** (for submission to the App Store). Click **Continue** to move to the next page.
4. On the 'Select an App ID' page, select the App ID from the list of App IDs you have already defined. Click the **Continue** button.
5. On the 'Select Certificates' page, select the certificate (Distribution) to include in the profile. Click the **Continue** button.
6. If creating an AdHoc provisioning profile:
On the 'Select Devices' page, check the device(s) that you want to be able to install and run the app. Click the **Continue** button.
7. On the 'Review, Name, and Generate' page, give the profile a name of your choice. It is helpful to include "App Store" or "AdHoc" in the name. Click the **Continue** button.
8. On the 'Download and Install' page, click **Download** to save your new .mobileprovision file to your computer.

This is the Provisioning Profile file that you will select in Reading App Builder.

Download existing mobile provision files

If other members of your team have already created provisioning profiles, they can be downloaded from the App Developer website by selecting the profile to be downloaded and pressing the **Download** button.

8. Building an iOS App

The first step in building an iOS app is to create a new app project following the instructions in the RAB **Building Apps** document. Then follow the instructions below.

8.1. Application builds available for iOS

For iOS distribution, three different app types are available.

8.1.1. Dedicated App

This is the standard traditional app and is the default. The content that is defined for this project will be used to generate an IPA file that may be delivered to the App Store for publication. A dedicated app could be viewed as a container app that only runs one preselected asset package that is included in the app at build time.

8.1.2. Container App

App Builder users have encountered issues with the Apple App Store reviewers if too many apps generated by the App Builder are published to the same account. Since the actual application differs only in the content provided, Apple rejects the app as being spam. To avoid these issues, a container app can be generated instead of the standard dedicated app.

The container app does not contain content of its own. It does not have book collections, audio or any of the other normal content. Instead, on its initial startup, it accesses a web site that is maintained by the app developer. The web site provides specially formatted links to multiple asset packages. The app user selects one of these asset packages. The container app downloads it and runs using the downloaded content. From this point, it resembles the dedicated app, running this set of content without further downloads being required.

The user of a container app does have the option of resetting the content at any time. If the user elects to reset the app content, the app asset package currently in use is deleted from his device. The web site accessed at the initial startup is displayed, and the user can once again select the asset package to be run from the available list. The selected package is then downloaded and run in place of the discarded one.

Building a container app will generate an IPA file similar to the dedicated app with the exception that no content is included with the app beyond some configuration information and a reference to the website used by the app. The IPA file is submitted to the Apple App Store in the same manner as the dedicated app.

While most of the settings associated with the app need to be set in the asset package projects, there are a few settings that must be set at the container app level. The first of these are the Firebase related features. Firebase tracks at the app level, and the settings for the *Firebase* tab and the *Security* tab will apply for all app asset packages associated with this container app. The `GoogleService-Info.plist` file that is used by the iOS Firebase configuration should be associated with the Package Name for the container app set on the package tab. Firebase will track information associated with the container app, not with the individual asset package projects. A project field is included in events generated by asset packages and sent to Firebase and can be used to identify the events that originated with a particular package that is being distributed.

Several other features follow the settings in the asset package project definition but must be accommodated by the container app that loads them. If any of the asset packages that can be loaded by this container app application use *Daily Reminder* features on the *Notifications* tab, these options must be enabled in the container app as well as in the asset package that wants to use the feature. The *IPA* tab also includes a checkbox that must be selected if any of the asset packages include audio files.

On the *Images* tab, the *iOS Icon* and the *iOS Splash Screen* (optional) should be configured for the container app as these will be the images used. The images set in asset packages for these two items will not be used.

8.1.3. Asset Package

Asset packages are used in conjunction with Container Apps. The configuration of an asset package project should be the same as it is for a dedicated app, setting the book collections, audio and feature required for that app. The only exceptions are those mentioned in the section above on Container Apps. However, where the build of Dedicated Apps or Container Apps results in an IPA file to submit to the Apple App Store, the build of an Asset Package project results in a zip file, saved to the IPA output folder, that is intended to be saved to a location where it can be referenced by the web site supporting the Container App. A user running who has downloaded the Container App from the App Store will select this package from the list presented by the Container App Web Site, at which point it will be downloaded to the device, decompressed and run as the project for the app. All of the book collection, audio, font, images, features and configuration information that is usually embedded within the dedicated app is saved in this zip file so that the project can be run as it would if it were configured as a dedicated app.

Dedicated Apps and Container Apps can only be generated on an Apple Mac device. Asset Packages do not require a Mac. Therefore, App Builder can build a project configured as an iOS Asset package on a Mac, a Linux device, or a Windows device. On a Mac Device, selecting *Build iOS App* for a project configured as an Asset Package will generate the zip file in the IPA Output Folder. For testing purposes, so that the app developer can test the device outside of the container app, selecting *Run iOS App in Simulator* will cause an IPA file to be generated in the Simulator Output Folder that will run using the project configuration. The app developer can load the IPA file on an iOS device simulator and test the project before generating the zip file and adding it to their Container App web site.

8.1.4. Container App Website

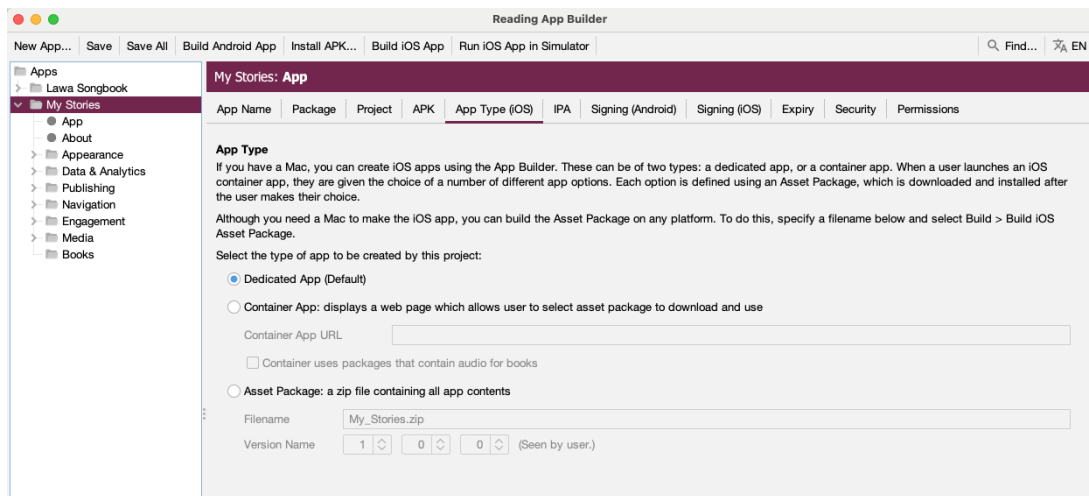
While not generated by App Builder, a Container App project must have a web site where the asset packages are located and a web page is defined by the user to allow selection of the asset package. The Container App displays the web page referenced by the URL that is part of its configuration. Anything other than links that reference

`asset://yourwebsite/yourassetpackage.zip` or [https://\[something\].zip](https://[something].zip) or [http://\[something\].zip](http://[something].zip) are passed through the browser and are displayed to the user. When the user selects a link that starts with `asset://` or a normal URL that ends in `.zip`, that is interpreted as the selection of an asset package. If the URL starts with `asset://`, the `asset://` in the link is replaced with `https://` to generate the destination URL. The file referenced is downloaded, unzipped, and used to initialize the Container App. A very simple example of a Container App Website web page would be:

```
<html>
<head>
<style>
.container {
  display: flex;
  justify-content: center;
  background-color: powderblue;
  font-size: 40px;
}
.center {
  width: 800px;
}
</style>
</head>
<body>
<p id="top" class="container">Container App Test</p>

<div class="container">
<ul> Select a translation
<li><a href="asset://sab-assets-test.s3.amazonaws.com/EnglishGreek2.zip">English
Greek</a></li>
<li><a href="https://Test.zip">Test</a></li>
</ul>
</div>
</body>
</html>
```

8.2. App Type (iOS) Tab

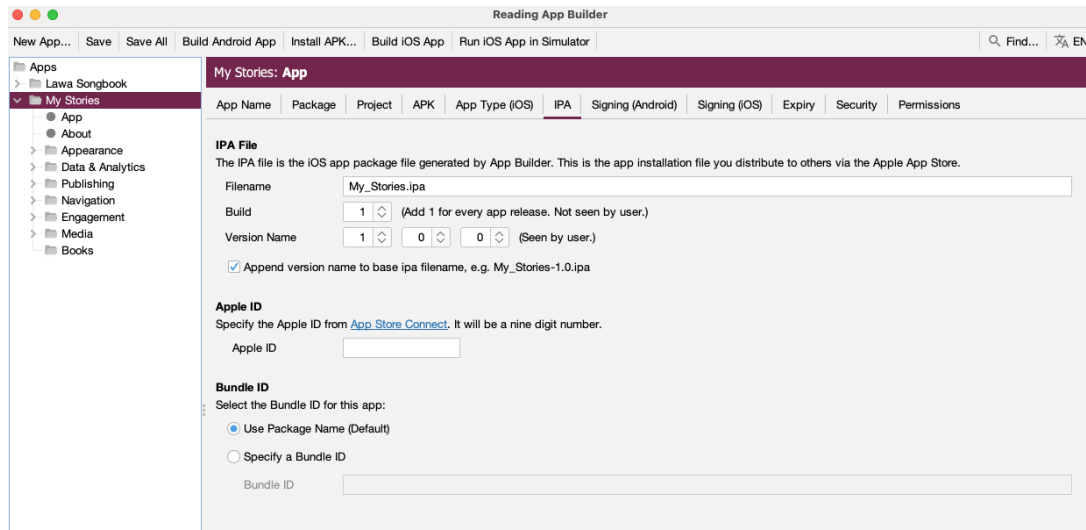


The **App Type** information identifies the type of application being built, as defined in the previous section. Select either *Dedicated App*, *Container App*, or *Asset Package*.

If the *Container App* option is selected, the **Container App URL** needs to be entered. The URL entered here is the web page that will be displayed to the user by the container app to allow them to select the asset package to be used. The checkbox underneath it must be selected if any asset package used by the Container App will contain audio for books.

If the *Asset Package* option is selected, the **Filename** field under the *Asset Package* option is enabled. This field defines the base name, without path, that will be assigned to the Asset Package zip file. The file will be created in the IPA Output folder when the user selects *Build iOS App* (or on a Linux or Windows system, selects *Build iOS App Asset Package*).

8.3. IPA Tab



Click on the **IPA** tab.

This allows you to set the name of the ipa file to be generated as well as the build and version information.

The remaining fields, under the **IPA File** and **Apple Id** headers are only enabled for a *Dedicated App* or *Container App*.

The **Filename** field on the screen for the **IPA File** section specifies the base name of the ipa file to be generated. If the checkbox at the bottom of the screen for *Append version name to ipa filename* is checked, then the version indicated by the **Version Name** fields is added to the base filename.

The **Build** field referenced as *Build* is also called *Bundle Version String*, *Bundle Version* or *CFBundleVersion* within Xcode and iTunes Connect. It represents the build number. The *Build* field expects an integer value and should be incremented with each file that is submitted to the iTunes Connect for release or testing.

The **Version Name** field is referenced as *Version*, *Bundle Short Version String*, *Bundle versions string, short* and *CFBundleShortVersionString* within Xcode and iTunes Connect. The field is created as a concatenation of the values of the three fields separated by a period. If the final field has a value of 0, then the version string is created from just the first two values. So for values of 1, 2 and 0, the resulting string is "1.2". For values of 1, 2 and 3, the resulting version string is "1.2.3".

The **Apple ID** field is the id assigned by App Store Connect to the application in the app store. It can be obtained from **Apple ID** filed on the App Information tab in the App Store Connect entry for the app as shown below

App Store Connect

Apps

Users and Access

 **Kuna Gospels** ▾

App Store

Features

TestFlight

iOS App

1.0 Prepare for Submission

Add macOS App

Add tvOS App

General

App Information

Pricing and Availability

App Privacy

Ratings and Reviews

Version History

In-App Purchases

Manage

App Store Promotions

App Information

This information is used for all platforms of this app. Any changes will be released with your next app version.

 You can now set your app's age rating from the App Information page. The Gambling and Contests content descriptors describe your app's content more accurately. [Edit Age Rating](#)

Localizable Information

Name ?

Kuna Gospels

18

Subtitle ?

30

General Information

Bundle ID ?

org.wycliffe.app.test.cuk

SKU ?

KUNA

Apple ID ?

1346657481

Primary Language ?

English (U.S.)

Category ?

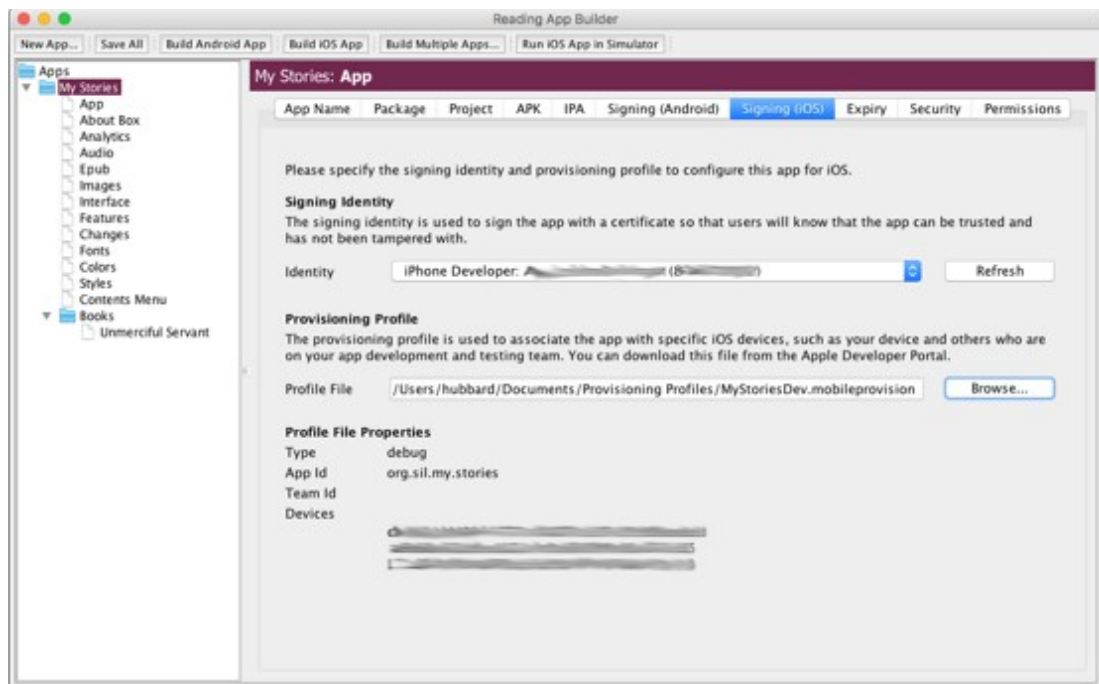
Primary

Secondary (optional)

The **Bundle ID** section allows the user to specify a Bundle ID for the app that is different than the package name. By default, the **Package Name** field from the **Package** tab is used as the Application Bundle ID that is used in creating the provisioning profiles and as the application identifier for the Apple App Store. If the user wants the iOS version of the app to have an identifier that is different than the package name that is used for the Android version of the app, press the **Specify a Bundle ID** button and then enter the ID to be used in the text box labelled **Bundle ID**.

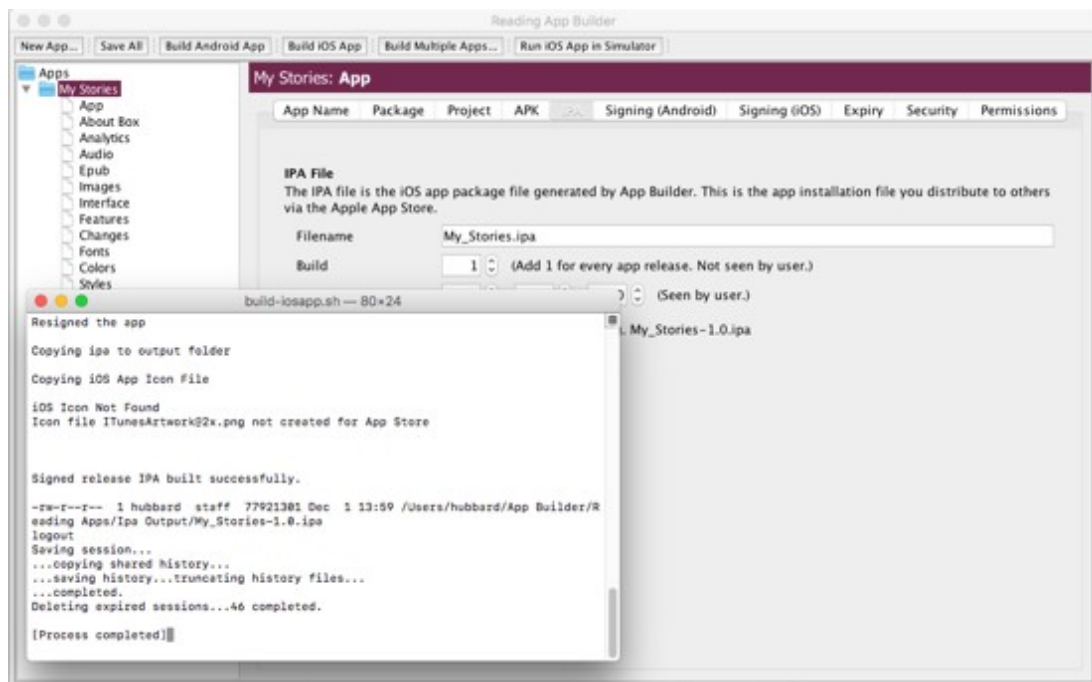
8.4. Signing (iOS) Tab

1. Select the **Signing (iOS)** tab to open the iOS signing options for the app.



2. Select the **Signing Identity** from the drop down list of signing certificates which have been downloaded and installed to this system in the earlier steps.
3. For the **Provisioning Profile** entry, enter or browse to the mobile provisioning file associated with the app that was downloaded in the earlier steps.

- Click on the **Build iOS App** button at the top of the screen. A terminal window should open. The build script for the iOS App should run within that terminal window.



- Examine the terminal window once the shell script has been completed. The message “Signed release IPA built successfully” should appear in the window if the app has been built successfully. (Note that occasionally the terminal window will appear behind the Scripture App Builder and that you have to select the terminal to review the results).
- The results of the build are an IPA file and an app that can be run in the simulator. They can be found in ~/App Builder/Reading Apps/Ipa Output/ and ~/App Builder/Reading Apps/Sim Output/.

Name	Date Modified	Size	Kind
Dictionary Apps	Oct 30, 2017, 10:11 AM	--	Fold
keystore	Nov 16, 2017, 3:30 PM	--	Fold
Reading Apps	Today, 2:01 PM	--	Fold
App Projects	Today, 1:50 PM	--	Fold
Ipa Output	Today, 1:59 PM	--	Fold
My_Stories-1.0.ipa	Today, 1:59 PM	77.9 MB	iOS /
Sim Output	Today, 2:02 PM	--	Fold
My_Stories-1.0	Today, 1:59 PM	57.2 MB	Appl
Scripture Apps	Nov 13, 2017, 12:16 PM	--	Fold

9. Testing an iOS App

After building the iOS IPA file, you will want to install it and test it on one or more devices before you submit it for publication to the Apple App Store. This manual describes two ways of doing this:

1. Use **Xcode** to install the IPA file to an iPhone or iPad that is connected to your computer.

This method is recommended if you have your test devices with you. It does not involve uploading and downloading the IPA to and from the internet.

2. Use **DeployGate** to upload the IPA file to the internet and share it with limited number of devices to download, install and test.

This method is recommended if you have good internet access and/or you have a team of testers who are elsewhere.

10. Using Xcode to Test an iOS App

To test your iOS IPA file using **Xcode**, do the following:

1. Launch **Xcode** and select **Window** ➤ **Devices and Simulators**.
2. Connect an iPhone or iPad to the Mac using a cable and unlock the device.
3. On the Mac, iTunes may launch and show a dialog asking “Do you want to allow this computer to access information...” Click on the **Cancel** button.
 - Note: This feature can be turned off in iTunes. Select **iTunes** ➤ **Preferences...**, select the **Devices** tab, and click on the **Prevent iPods, iPhone, and iPad from syncing automatically** checkbox.
4. On the device, it may prompt to **Trust This Computer**. Tap on the **Trust** button. This may require to enter the Passcode to trust this computer. The device will show up in the **Xcode Devices** window.
5. The first time the device is connected to the Mac, Xcode will take some time to Prepare debugger support. This may take some time. Please wait.
6. In the **Xcode Devices** window, there will be a section named **INSTALLED APPS**. Click on the + button.
7. Find the IPA file to add. Click **Open** after you have selected it.
8. Click the **Install** button next to the name of the app.
9. Wait until the Mac installs the app to your device.

After the install is complete, you will see the app icon on your device and you can test it.

11. Using DeployGate to Test an iOS App

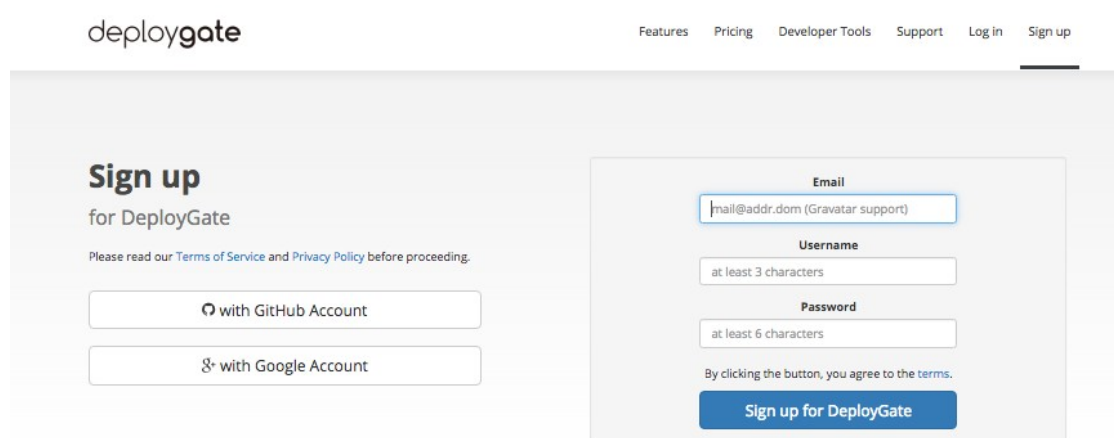
DeployGate enables you to test your app and share it with a limited number of users to test. You upload the IPA file for iOS apps or the APK for Android apps and then download them to your phone or tablet device. You can also invite testers to install your app and help with the testing.

A DeployGate app is installed on the testing device. It will show all of the apps that you have uploaded to DeployGate and allows them to be installed on the device.

11.1. Creating a DeployGate Account

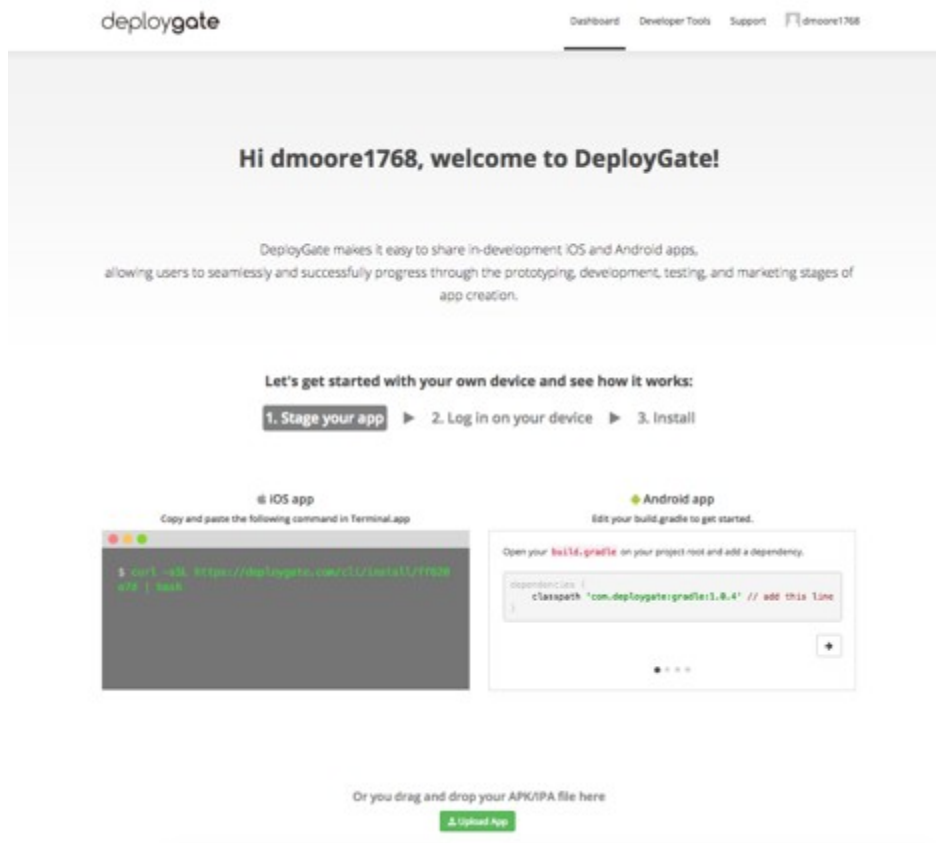
The first step is to create a DeployGate account. To do this:

1. Go to <https://deploygate.com>
2. Press the **Get Started** button.
3. Enter an email address, a user name and a password.
4. Press the **Sign up for DeployGate** button.

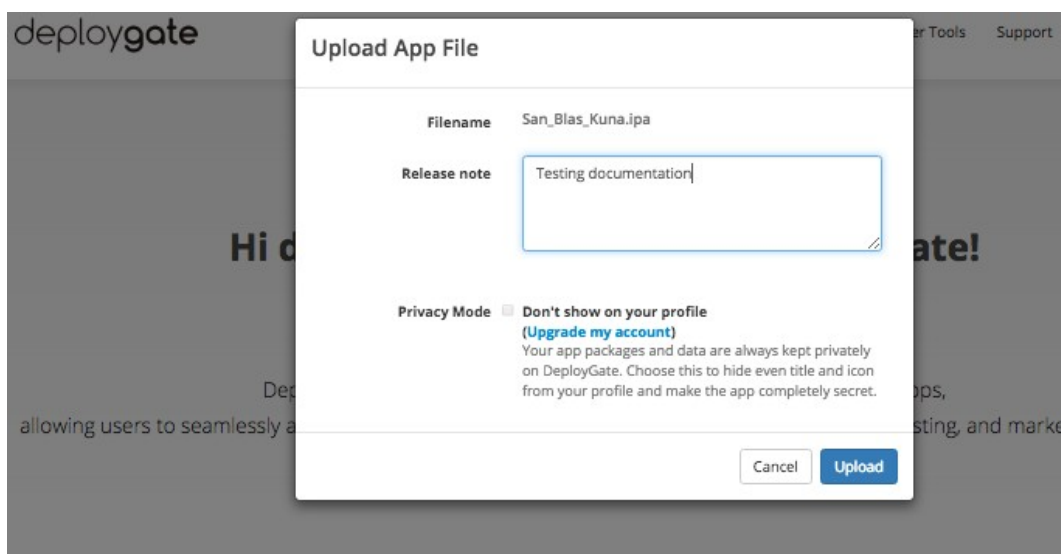


11.2. Uploading your first app

Once the sign in screen has been successfully completed, you are presented with a screen that prompts you to upload your app. While there are several methods described for embedding it as part of your build process, the way we have used this to date is to simply upload the IPA file that has been created by locating the ipa file in Finder and then dragging it to the bottom area of the screen where it has a green **Upload App** area:

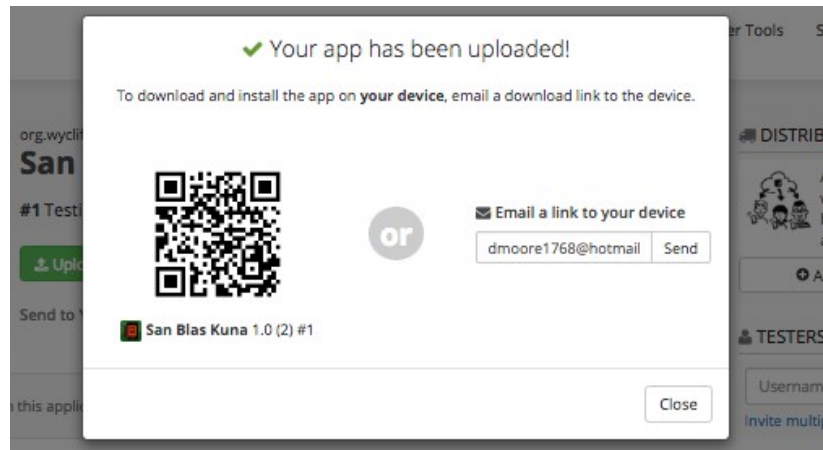


Dragging the file to the upload area causes a new dialog to be displayed with the name of the file and a text box where you can enter a short note that will be displayed on your profile window and also on the DeployGate app when the user is selecting the app. It is a good place to write a short note on the reason for the update so that it is easy for the testers to see that the app has been updated and to see what the primary reason they need to update is. Complete the screen and press the **Upload** button.

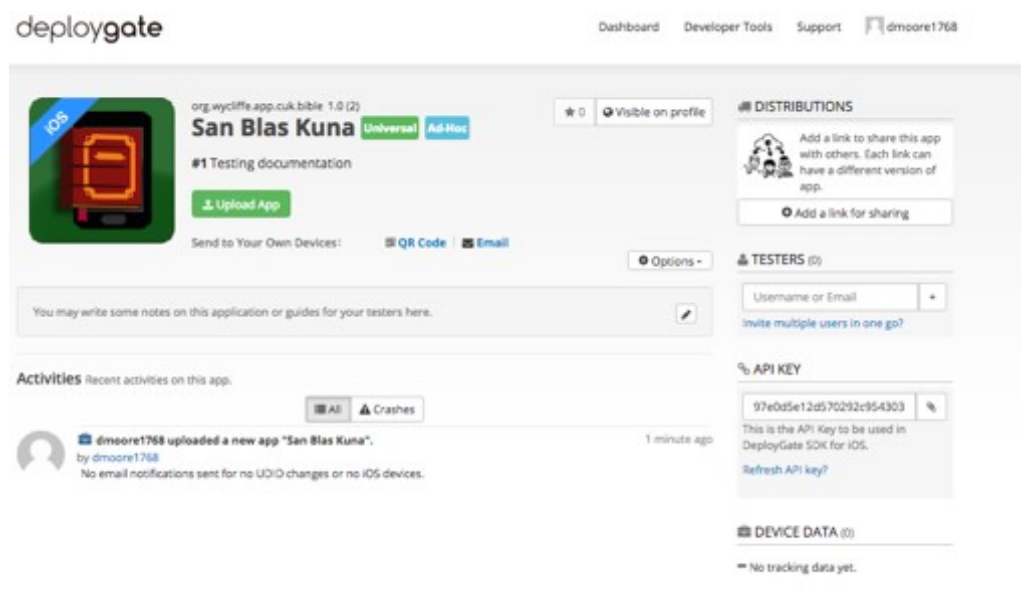


When the upload is complete, a new dialog is displayed with a QCode bar code and the option to send an email to your device. The QCode can be read in with your iPhone or iPad which will trigger an installation of the DeployGate app using your app profile.

Alternatively, you can enter an email address at this point, which you would also open on the iPhone to install the DeployGate app with the correct profile. Or you can simply go on and add users and devices later.



After this, the screen that is displayed is what you will normally see when you login. The screen has an entry for each app that you have uploaded. It has an **Upload App** button that can be used to upload new versions of the same app or to upload a new app.



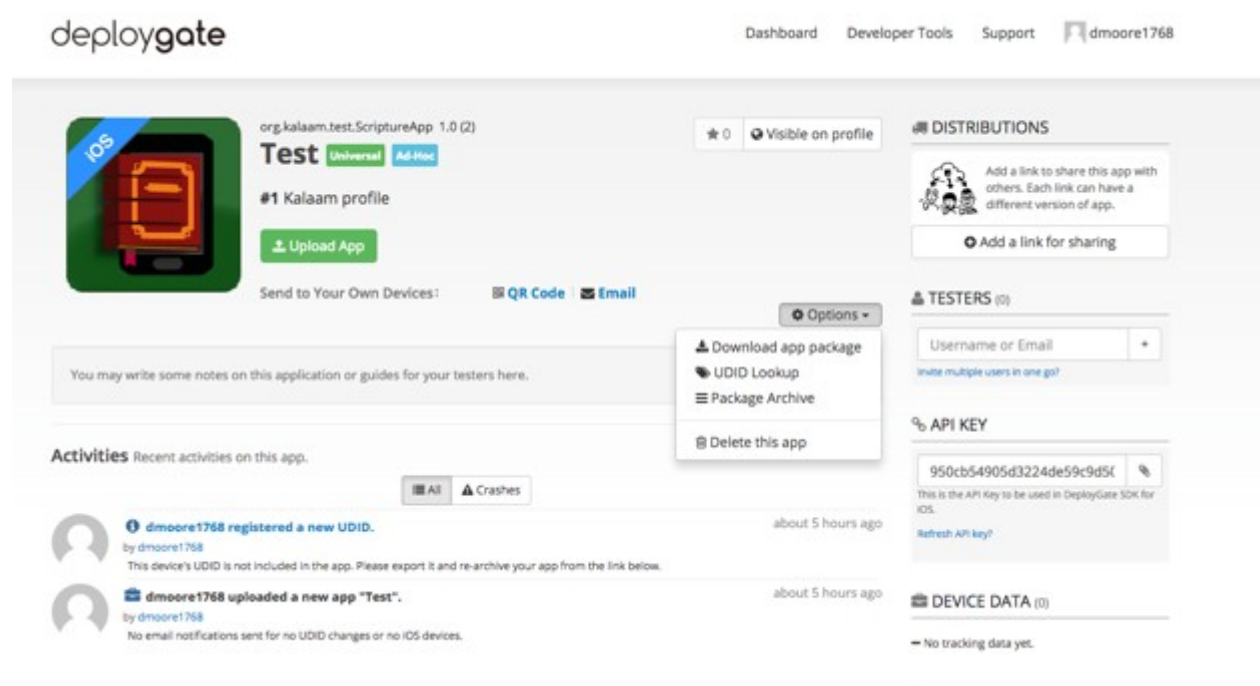
11.3. Registering a Device

If the iOS device was not originally in the mobile provisioning profile and if the device has not been previously registered in your Apple Developer account, you need to add it to both.

There is a method for manually doing this, but DeployGate provides a way of simplifying the process so that you don't need to go and look up UDID for the device.

First, try to install the app using the method described above. You will not be able to install it because the UDID is not registered for the app. However this will result in the device being registered in DeployGate which allows the following steps.

After attempting to install the app, re-enter DeployGate in your browser and open the entry for your app. As you can see in the screen below, it will show that a new UDID has been registered for the device. Press the **Options** button below and select the **Package Archive** option. Next click the little tag symbol inside the app box to open the UDID list.



The next screen shows a list of the devices that have been observed by DeployGate or that were included in the provisioning profile. Your new device entry will show up on the screen with a **Not Exist** entry. Make sure that the entry for the new device is checked and then press the **Export Selected UDIDs** button. This will create a file "multiple-device-upload-ios.txt" that can be used on the Apple Developer website to add these devices to the mobile provisioning file.

**Test** by dmoore1768[Test](#) / [Package Archive](#) / UDID List of Members and Revision #1

If some UDIDs are **"Not Exist"** in the profile, this app can't be installed to those devices.
In that case, export UDIDs as a file, then import it at **"Certificates, Identifiers & Profiles"**, re-create a profile with those devices.

[Export Selected UDIDs](#)

Show 10 entries

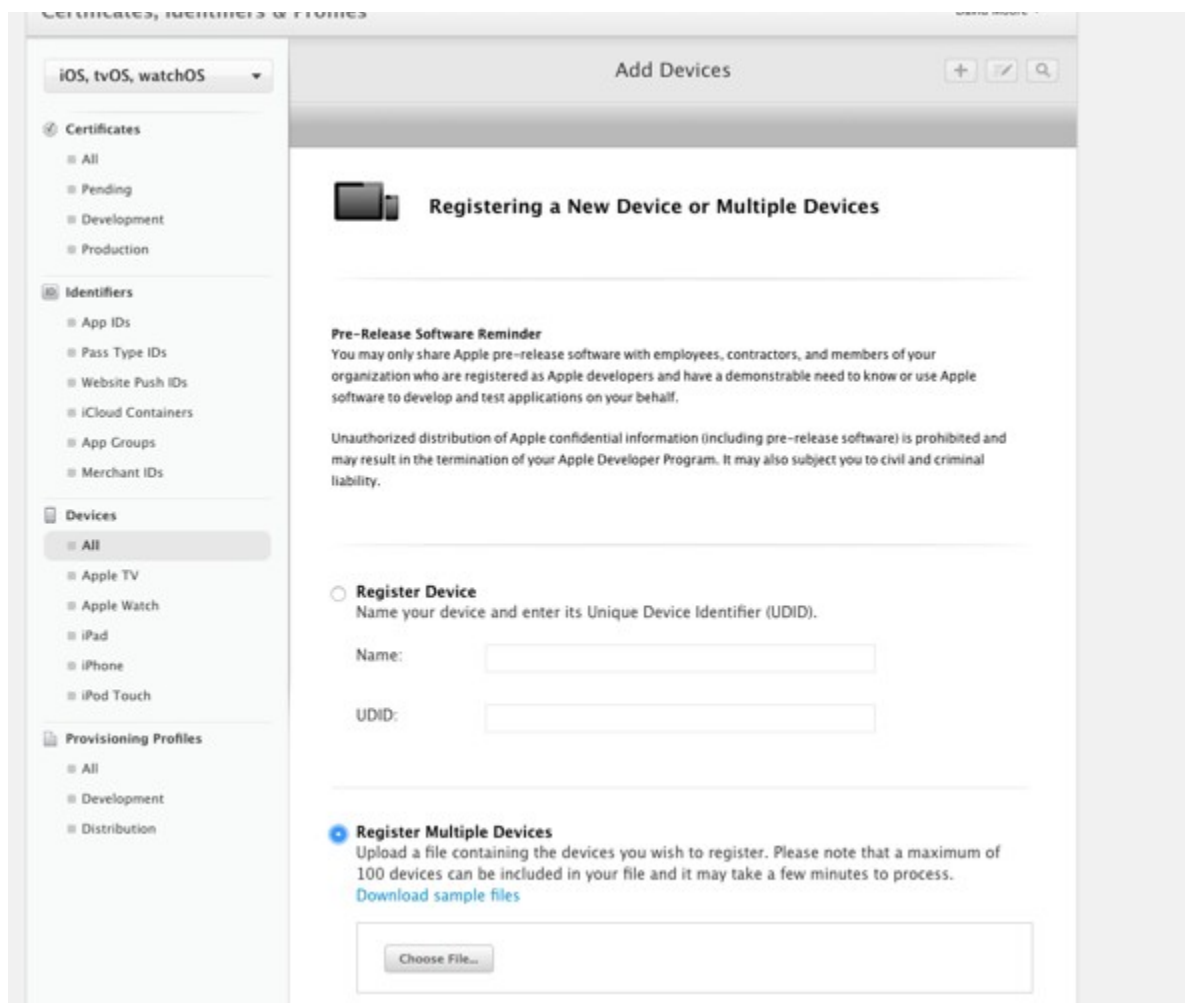
Search:

	USER	ROLE	PRODUCT	UDID	PROVISIONING PROFILE
☒	dmoore1768	Developer	iPad 3 (WiFi)	9f0a0d2b9979a47a5d96bb1a1369e0db545ad917	Not Exist
☐		-	iPhone 5 (Global)	3c237054805042594f6d8814c7972f931e4ec5b2	Exist

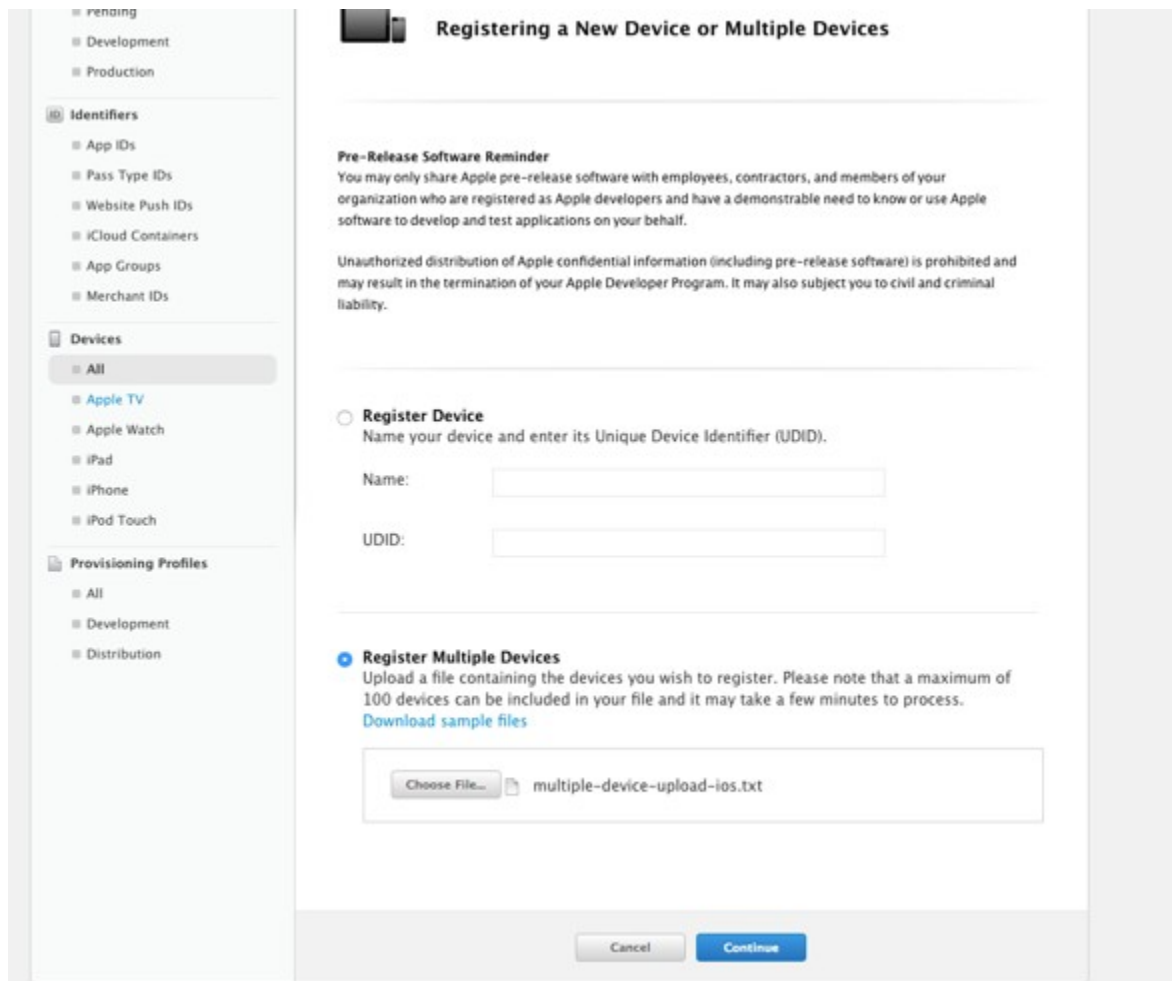
Showing 1 to 2 of 2 entries

[Previous](#) [1](#) [Next](#)

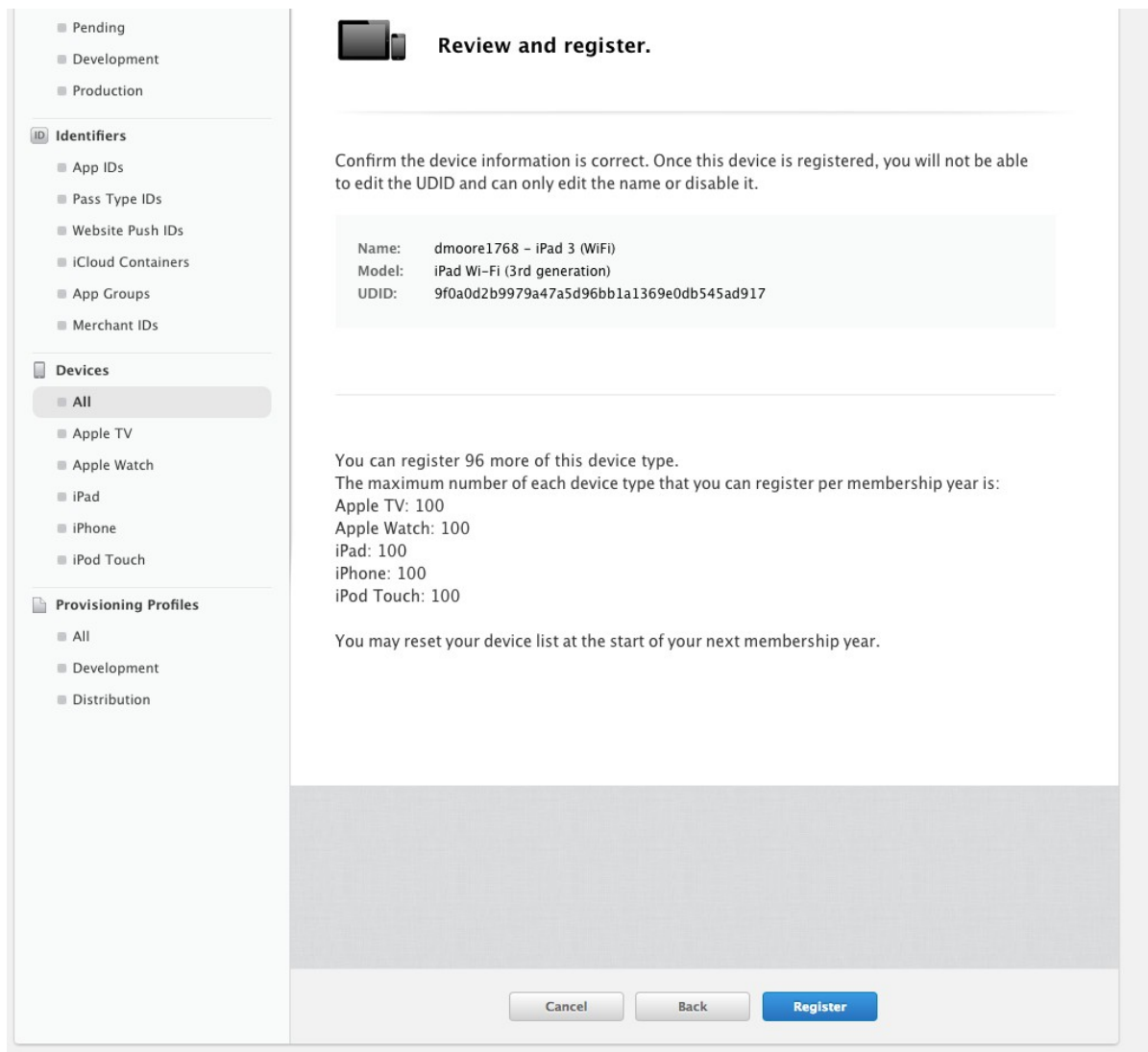
After logging in to Apple Developer, click on **Certificates, IDs and Profiles**. Press the **All** selection under **Devices** as shown in the illustration below and then select **Register Multiple Devices**. Then press the **Choose File** button.



Find the multiple-device-upload-ios.txt file that was created by DeployGate and then press **Continue**.

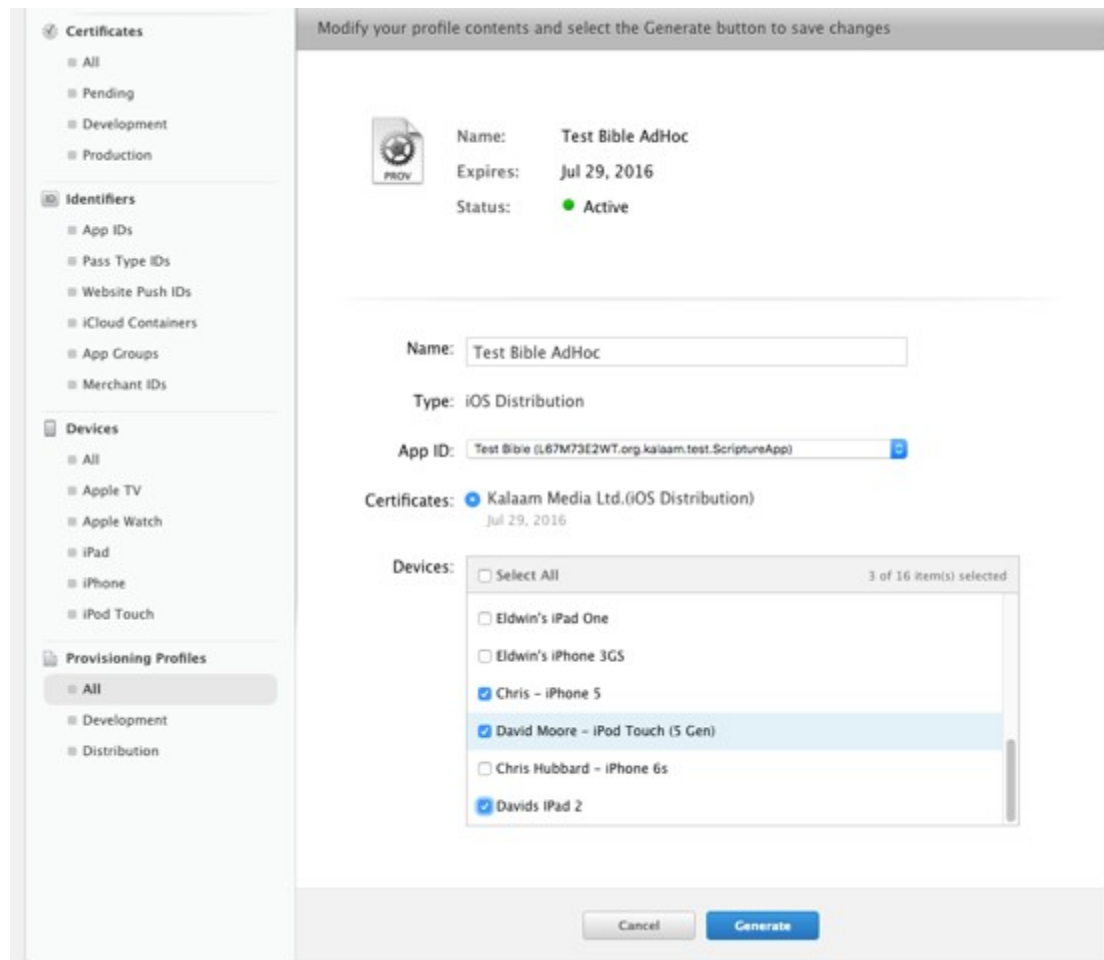


A review screen will be displayed which should list the name to be assigned to the device along with the UDID associated with the device. Review to ensure this is correct and press **Register**.

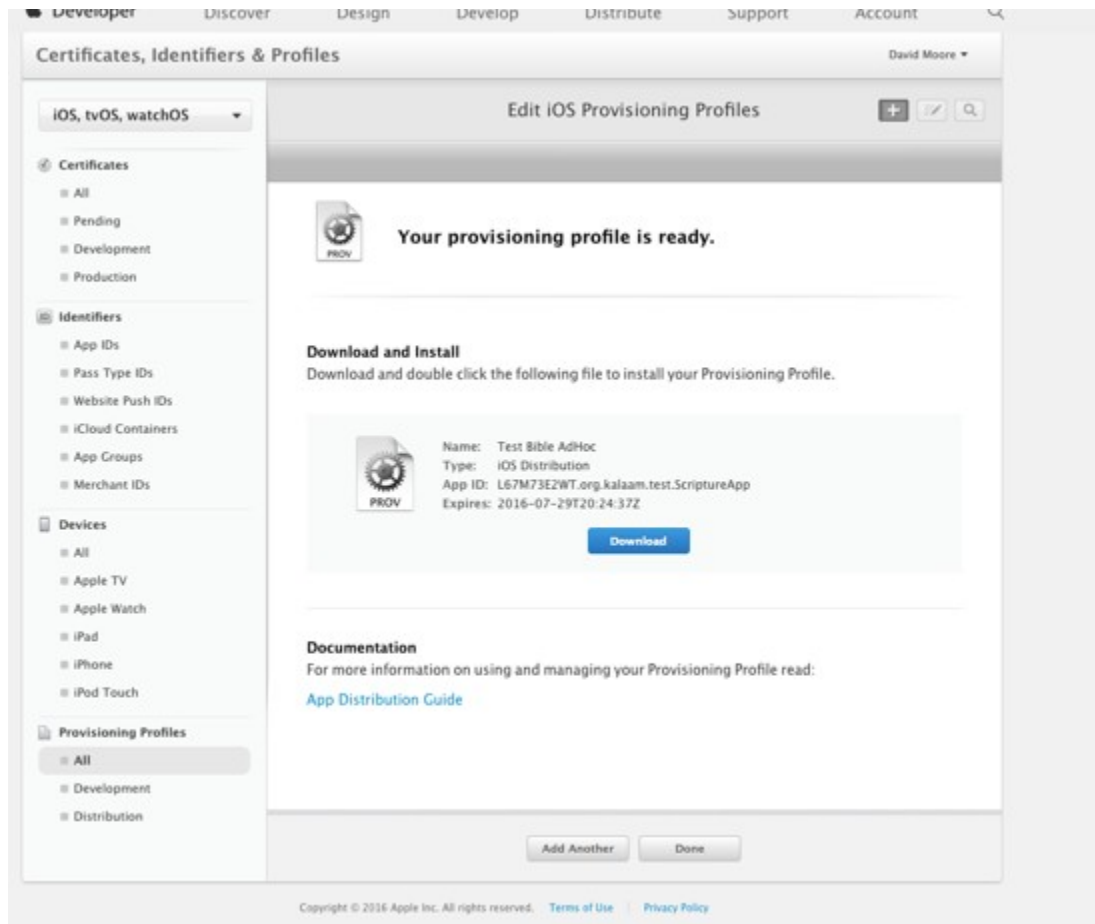


At this point your device has been registered to your Apple Developer account. You now need to add the device to the provisioning profile for your app. Select the provisioning profile being used to test the app. Make sure your test device is checked in the list of devices at the bottom of the screen. Press **Generate**.

Note: For the purposes of testing with DeployGate, an AdHoc type of provision profile must be created and used. If you have not selected AdHoc, the list of devices will not be available on the screen.



The following screen will display:



Now you can download the new provisioning profile that has been generated.

Next you need to rebuild the iOS app using the new profile and upload it to DeployGate again. This time when you attempt to install it, the **Install** button should be enabled and will allow you to install your app.

12. Uploading an iOS App to Apple App Store

Before attempting to upload the app, you will need to create an entry in App Store Connect (<https://appstoreconnect.apple.com>).

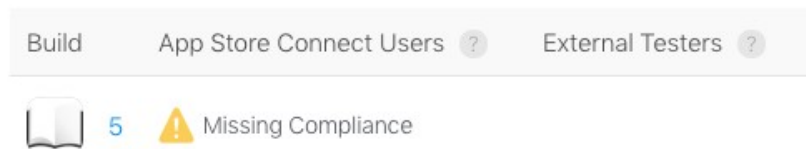
Select the appropriate distribution **Signing Identity** and **App Store** provisioning profile on the **Signing (iOS)** tab and click Build iOS App. This will create an IPA file in the IPA Output Folder.

Launch **Transporter** and click **Sign In** using the Apple ID for your Apple Developer Account. Drag and drop the IPA file from the IPA Output Folder to Transporter. Once the app is added to Transporter, click on the **Deliver** button. After it is uploaded, you can click on the ... button and select **View in App Store Connect**. Selecting the Activity tab will show the status of the processing of the upload.

13. Using Test Flight to Test an iOS App

It will take a little bit of time (around 20 minutes) for the app to be processed in App Store Connect. You will receive an email when the app is done processing. Switching to the Test Flight tab in App Store Connect, you will see that app is Missing Compliance.

▼ Version 1.4



Click on the build number and you will be taken to a page where you can click on **Provide Export Compliance Information**. The app uses an encryption algorithm to protect the text of the app.

Click **Yes** to the first dialog that the app uses encryption and then click **Next**.

Export Compliance Information

Does your app use encryption? Select Yes even if your app only uses the standard encryption within Apple's operating system.

☐ Yes
☐ No

Click **Yes** to the second dialog to indicate that the app qualifies for an exemption due to **(b) Limited to intellectual property and copyright protection** and then click **Start Internal Testing**.

Export Compliance Information

Does your app qualify for any of the exemptions provided in Category 5, Part 2 of the U.S. Export Administration Regulations?

☐ Yes
☐ No

Make sure that your app meets the criteria of the exemption listed below. You are responsible for the proper classification of your product. Incorrectly classifying your app may lead to you being in violation of U.S. export laws and could make you subject to penalties, including your app being removed from the App Store.

You can select Yes for this question if the encryption of your app is:

- (a) Specially designed for medical end-use
- (b) Limited to intellectual property and copyright protection
- (c) Limited to authentication, digital signature, or the decryption of data or files
- (d) Specially designed and limited for banking use or "money transactions"; or
- (e) Limited to "fixed" data compression or coding techniques

You can add App Store Connect users (normally users in your organization) to test the app. There is a link at the left for **Add External Testers**. This will require the app to go through Beta App Review.

14. Apple Privacy Policy

Prior to publishing your app through **App Store Connect**, you will be required to complete the privacy policy details section. The answers to these questions will be used to create the privacy policy section in your app store entry that describes to the user how you are using their data. The types of data associated with each question is documented by Apple at <https://developer.apple.com/app-store/app-privacy-details/>. The answers associated with the SAB app depend upon the use of analytics and what data is saved if you have user registration defined for your app.

14.1. Data Types

Under the initial *Data Collection* screen in this section, if the app does not use analytics or user registration, you may indicate that the app does not collect any data. If either of these features are in use, then you should respond that we do collect data from this app. If you answer Yes to this question, then you need to specify in the next screen the types of data collected. For an app that has analytics enabled, *Product Interaction* under *Usage Data* and *Crash Data* under *Diagnostics* should be selected. If the app has User Registration enabled, then some of the fields under *Contact Info* may need to be checked depending upon the fields you have configured for the user to enter.

14.2. Product Interaction

This section should only be required if analytics or user registration are enabled. If analytics is enabled, then the *Analytics* entry should be checked to indicate that the data is being used to track user behavior. If user registration is enabled, you may also want to check the *App Functionality* entry. On the next screen, you should check the “No, product interaction data collected from this app is not linked to the user’s identity” entry. On the final screen, regarding tracking, you may click the “No, we do not product interaction data for tracking purposes”.

15. Building from Terminal

Reading App Builder has a command line interface which allows you to create a new app and build it or load an existing app and build it.

15.1. Installing Command Line Interface

Choose the **Tools > Install rab command in PATH** menu item to enable the use of the command line interface. You will be prompted to enter your login password to install the command.

15.2. Command Line Interface Options

Run the following command to run the Reading App Builder application followed by a series of options described below. The command is:

`rab`

The available parameters are:

Option	Description
-new	Create a new app project
-load <project>	Load an existing app project
-build	Build app project (use with either -new or -load)
-no-save	Do not save changes to app (use with -load)
-resign	Resign iOS Template App (use with either -new or -load)
-?	Show command line help
-n <app-name>	Set app name. Enclose the name in "double quotes" if it contains spaces.
-p <package-name>	Set package name, e.g. com.myorg.language.appname
-b <filename>	Add book or bundle file. This could be a USFM file or a zipped set of USFM files. It could also be a Digital Bible Library text release bundle.
-i <filename>	Include additional parameters file. Use the full path of the file and enclose it in "double quotes" if there is a space in the path.
-a <filename>	Set about box text, contained in text file. Use the full path of the file and enclose it in "double quotes" if there is a space in the path.

-f <fontname>	Set font name or filename, e.g. "Charis SIL Compact", "c:\fonts\myfont.ttf" The font name must be one of the items in the list of fonts in the New App wizard. For other fonts, specify the full path to the font filename.
-g	Use Grandroid
-ic <filename>	Add launcher icon (one or more .png files). Use the full path of the files and enclose them in "double quotes" if there is a space in the path.
-l <lang-code>	Set language for menu items and settings, e.g. en, fr, es
-ft <feature=value>	Set a feature, e.g. book-select=grid
-vc <integer>	Set version code, e.g. 1, 2, 3, or +1 to increment the current version code by 1.
-vn <string>	Set version name, e.g. 1.0, 2.1.4, or use +1, +0.1, +0.0.1 to increment the current value.
-ks <filename>	Set keystore filename. Use the full path of the file and enclose it in "double quotes" if there is a space in the path.
-ksp <password>	Set keystore password
-ka <alias>	Set key alias
-kap <password>	Set key alias password
-fp <folder=path>	Set a folder path, e.g. "app.builder=/Developer/Reading App Builder".
-ta <target-api>	Set Target API, e.g. 21 for Android 5.0, 22 for Android 5.1.
-si <signing identity>	Set Signing Identity to use for iOS Resigning
-pp <provisioning profile>	Set full path to provisioning profile for iOS resigning
-bn <integer>	Set build number for ipa file, e.g. 1, 2, 3, or +1 to increment by 1
-vs <string>	Set version string for ipa file, e.g. 1.0, 2.1.4 or +1, +0.1,

	+0.0.1
--	--------

Examples:

```
rab -load "Mali" -resign -bn "5" -vs "2.3.2" -si "iPhone Distribution: Summer
Institute of Linguistics, Inc (SIL) (4YF5X97M4H)" -pp
"/Users/builder/Documents/MobileProvision/AdHoc_org.wycliffe.app.mali.mobileprovisi
on"
```

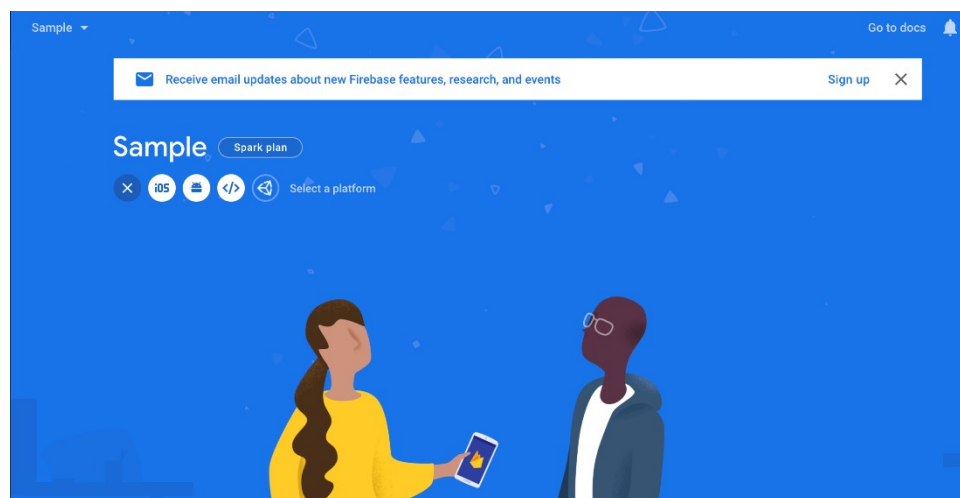
Running the command line with no parameters will start the Reading App Builder UI from the command line.

16. Using Firebase in an iOS App

The *Building Apps* document contains general information about setting up the app to support Firebase Analytics, Crashlytics, Messaging and Real Time Database. This section will focus on the steps required to make these features work with an iOS app.

16.1. Adding an iOS App

The first step is to go to your Firebase Console (<https://console.firebase.google.com/u/0/>) and select the entry for the application you are working on. Press the “Add App” button, then select the iOS button.



The next screen will request general information about your application. The iOS bundle ID should match the Bundle ID for the app. This is the SAB Package setting from the Package Tab for the App unless a separate Bundle ID is set in the IPA Tab . The App Nickname can be set to whatever you want this app to be referenced as within Firebase. The App Store ID is the same as the Apple ID field in the SAB IPA tab described in Section 9. Once these fields are entered press “Register App”.

× **Add Firebase to your iOS app**

1

Register app

iOS bundle ID ⓘ
org.sil.sample

App nickname (optional) ⓘ
Sample App

App Store ID (optional) ⓘ
1234567890

Register app

2

Download config file

3

Add Firebase SDK

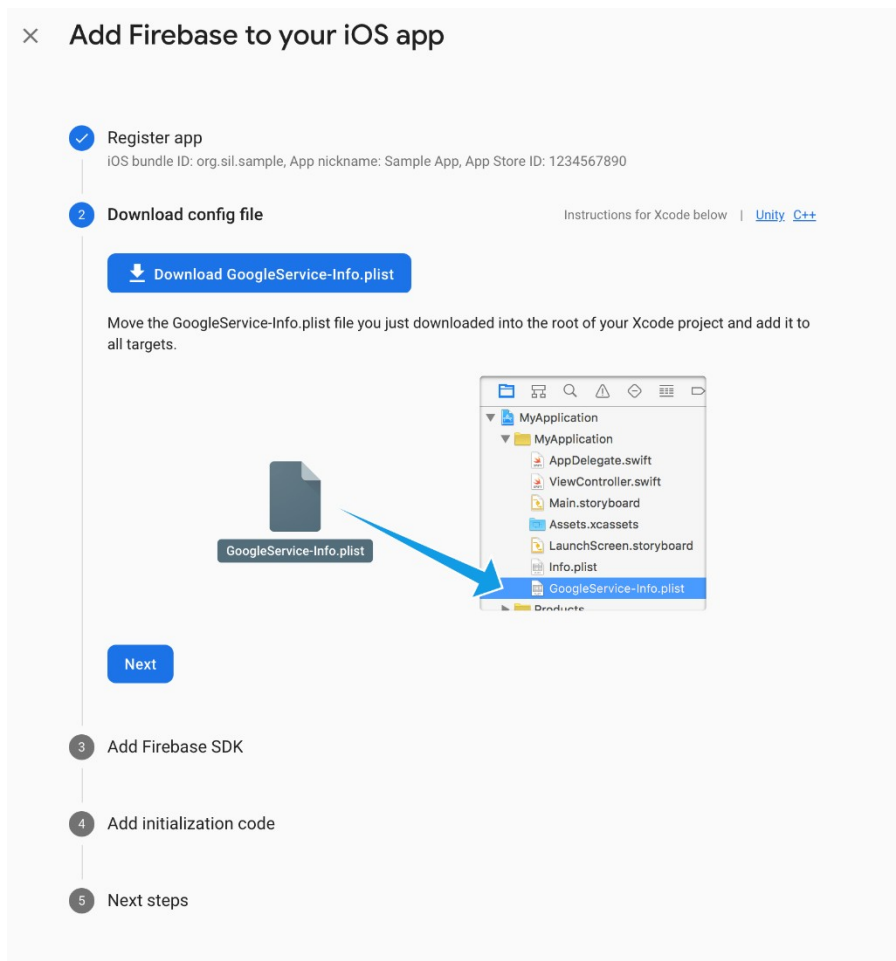
4

Add initialization code

5

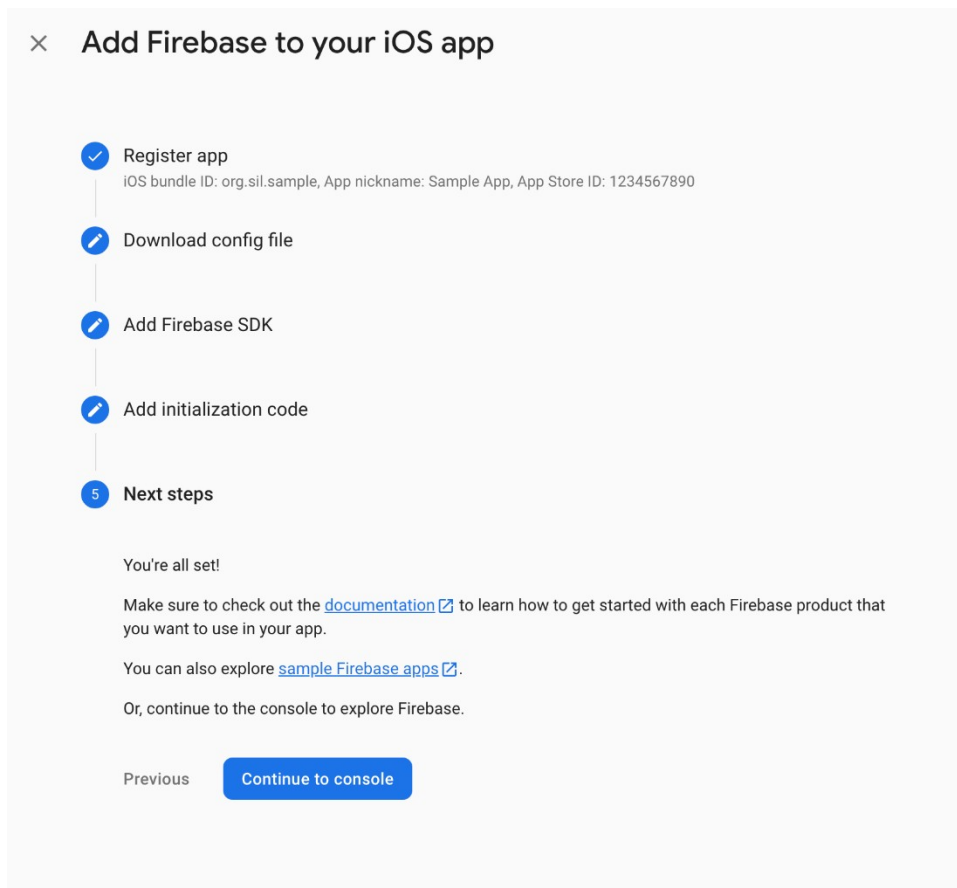
Next steps

The next screen allows you to download the config file for the application. Press “Download GoogleServiceInfo.plist” to download the Firebase configuration file to your local machine. It should be mentioned here that if your app has the Real Time Database option set in RAB and you have not yet configured the Real Time Database within Firebase Console, you should either do that first or download the plist file again after that configuration has been completed. Download the file to a location where you can locate it. Later in this section, instructions will be provided for adding this file into your RAB application. After downloading the file, press “Next” to continue.



The screens which follow the download step are all instructional screens providing information about programming changes that need to be made to add Firebase to an app. This work has already been done within RAB and can be ignored here.

The final screen just provides a means to return to the console. The addition of the iOS app to the Firebase console is complete.



16.2. iOS Configuration for Firebase in SAB

After the app has been added to the Firebase configuration through Firebase Console, the configuration file that was downloaded needs to be added to SAB. On the **Firebase** tab, in the **Firebase Configuration** section on the bottom half of the screen, press the **iOS** tab. Press the **Browse** button to locate the *GoogleService-Info.plist* file that was downloaded during the Firebase configuration process above. Select that file to add it to the app configuration. Note that if Firebase features are checked as enabled, the iOS app will not build until the plist file has been added.

Build Multiple Apps... Run iOS App in Simulator Install APK...

My Stories: **App**

App Name Package Project APK IPA Signing (Android) Signing (iOS) Expiry **Firebase** Deep Linking Security Permissions

Firestore can provide a range of tools for your app, such as analytics, crash reporting and push notifications. To use Firebase in your app, [create a Firebase project](#) for the app and specify the configuration details below.

Firebase Features
Which Firebase features would you like to use in this app?

- ☒ Firebase Analytics (for tracking app usage and user engagement)
- ☒ Firebase Crashlytics (for crash reporting)
- ☒ Firebase Messaging (for push notifications)
- ☒ Firebase Realtime Database (for saving user details)

Firebase Configuration
Android **iOS** Web

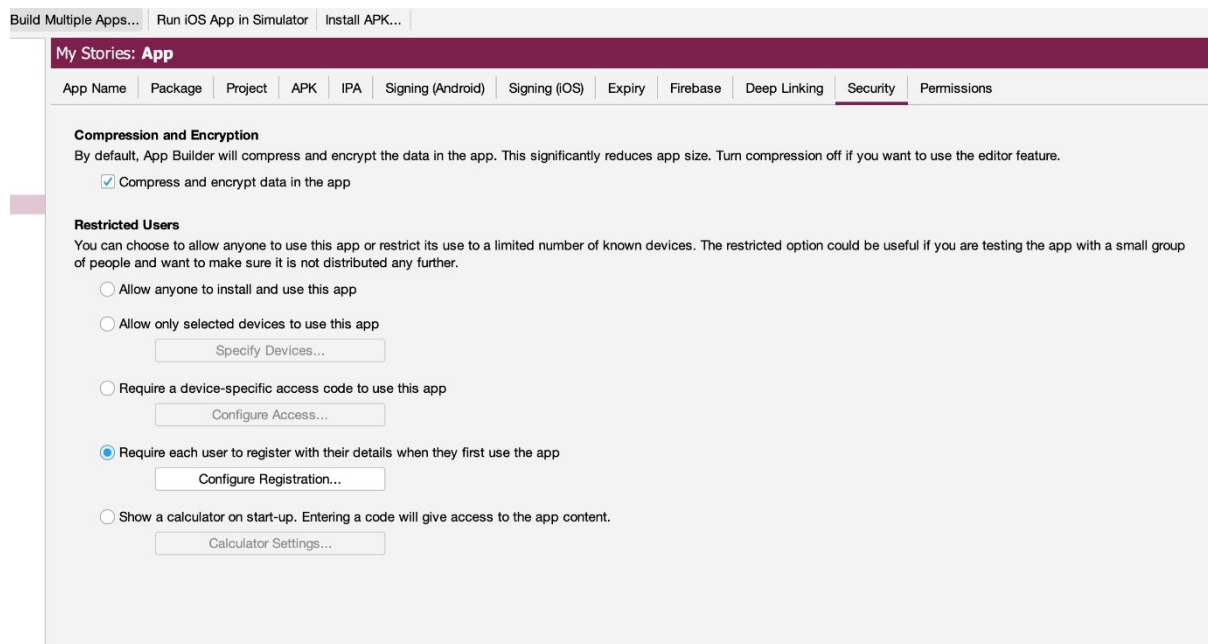
Specify the Firebase configuration file for iOS (GoogleService-Info.plist). This can be downloaded from your Firebase Project Settings page.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>CLIENT_ID</key>
  <string>516797509905-f82an4nbkl4s1buvi03ate1sb2iltei.apps.googleusercontent.com</string>
```

Browse...
Clear
Open Console...

16.3. Security Feature Support in iOS App

The iOS app supports a subset of the security features available in RAB. These features can be accessed through the **Security** tab within RAB. iOS only supports the default *Allow anyone to use this app* and the *Require each user to register with the details when they first use the app* features. The other features are available for Android apps but are not implemented in the iOS app at this time. The registration configuration for this feature works identically for both Android and iOS. No special steps have to be taken specifically for iOS to configure this option if it is selected.



16.4. Firebase Messaging

If Firebase Cloud Messaging is configured to allow push notifications for the app, several changes are required for the app's configuration information in the Apple App Store. The first thing that is required is that the App's configuration must be changed in Apple Developer to include Push Notifications. Under *Certificates, Identifiers, and Profiles*, select the app that is being configured. Enable the Push Notifications option from the list.

☐	 NFC Tag Reading ⓘ	
☐	 Personal VPN ⓘ	
☒	 Push Notifications ⓘ	Configure Certificates (0)
☐	 Sign In with Apple ⓘ	Configure
☐	 SiriKit ⓘ	
☐	 System Extension ⓘ	
☐	 Time Sensitive Notifications ⓘ	
☐	 User Management ⓘ	
☐	 Wallet ⓘ	
☐	 Wireless Accessory Configuration ⓘ	

After making these changes, the provisioning profile for the app should be updated for the new capability and the entry updated in SAB.

For Firebase to send notification to the app, it will need to have a key uploaded to it. The preferred manner is to go to the **Keys** section in Apple Developer and select to create an Apple Push Notifications service key, using the first option on the screen below. One key of this type is created to be used with all apps associated with this organization. If a key has already been created, then use that.


David Moore ▾
Summer Institute of Linguistics, Inc (SIL) - 3YE4W86L3G

Certificates, Identifiers & Profiles

[< All Keys](#)

Register a New Key

[Continue](#)

Key Name

You cannot use special characters such as @, &, *, ' ", -, .

ENABLE	NAME	DESCRIPTION	
☐	Apple Push Notifications service (APNs)	Establish connectivity between your notification server and the Apple Push Notification service. One key is used for all of your apps. Learn more	
☐	DeviceCheck	Access the DeviceCheck and AppAttest APIs to get data that your associated server can use in its business logic to protect your business while maintaining user privacy. Learn more	
☐	MapKit JS	Use Apple Maps on your websites. Show a map, display search results, provide directions, and more. Learn more	Configure
☐	Media Services (MusicKit, ShazamKit)	Access the Apple Music catalog and make personalized requests for authorized users, and check audio signatures against the Shazam music catalog. ⓘ There are no identifiers available that can be associated with the key	Configure
☐	Sign in with Apple	Enable your apps to allow users to authenticate in your application with their Apple ID. Configuration is required to enable this feature. ⓘ There are no identifiers available that can be associated with the key	Configure
☐	ClassKit Catalog	Publish all of your ClassKit app activities to teachers creating Handouts in Apple Schoolwork. Learn more	

At the time the key is created, a .p8 file is generated which can be used for all apps. You are allowed to make two keys through this interface, so that you can create a new key and revoke the old one, but only one is active at a time.

Certificates, Identifiers & Profiles

[< All Keys](#)

View Key Details

Download

Revoke

Edit

Name

SIL APNS

Key ID

55438BX8MT

Enabled Services

NAME

CONFIGURATION

Apple Push Notifications service (APNs)

Once the p8 file for this key has been downloaded, the file can be uploaded to the Firebase configuration for cloud messaging for this app.

Project settings

General

Cloud Messaging

Integrations

Service accounts

Data privacy

Users and permissions

App Check BETA

Project credentials

Add server key

Key

Token

Server key

AAAAVZW_IqlAPA91bHGkdH5CPgl1IU5JEx-W-1PugA7_3y10JBjAlHmMzeynfJeLi2ldmakKnnzLm6BnpjXzHgkEN3GwGuQwZRD0ygMPJY_mGjCgWk5AoEEvM-9ubRFTdQ5ucvQpNg0lF9V2_01

Sender ID ⓘ

367584581282

iOS app configuration

iOS apps



English Greek

org.wycliffe.app.englishgreek2

Firebase Cloud Messaging can use either an APNs authentication key or APNs certificate to connect with APNs

APNs Authentication Key

File

Key ID

Team ID

APNs Auth Key

55438BX8MT

3YE4W86L3G

Delete

APNs Certificates

16.5. Firebase Crashlytics for iOS

If Crashlytics is configured for the app in Firebase, the dSYM file associated with the app should be uploaded to get the detailed information about any crashes. To upload dSYMs, you'll need to use the `upload-symbols` command line tool that ships with the Crashlytics SDK. One way to obtain the tool is to install the Crashlytics SDK and locate the tool in the

“FirebaseCrashlytics” folder where you installed the SDK. The tool has also been uploaded to an App Builder share drive. The link to this file is:

<https://drive.google.com/file/d/1RJIDZyQUbfhogg9IGZg1YggNMJQQVeue/view?usp=sharing>

After downloading this file, cd to the download location and enter “chmod +x upload-symbols” to make the script executable. Otherwise a “Permission denied” may be encountered when attempting to run the script.

The dSYM file that is required is supplied as part of the build process if Crashlytics is configured for the application. The dSYM file is created in the ipa output folder with the same name as the ipa file, with a dSYM extension.

Upload the dSYM file using the *GoogleService-Info.plist* file associated with this application by opening a Terminal window. The command line to upload the dSYM file is:

```
/path/to/upload-symbols -gsp /path/to/GoogleService-Info.plist -p ios  
/path/to/TemplateApp.app.dSYM
```

For the example where all three files have been placed in the same directory and the Terminal window has cd'd to that directory, the command would be:

```
./upload-symbols -gsp ./GoogleService-Info.plist -p ios ./TemplateApp.app.dSYM
```